

xv6-labs-2025 操作系统实验报告

课程名称： 操作系统课程设计

课号： 42028704

姓名： 刘相成

学号： 2452207

代码仓库： <https://github.com/goagain-qidong/xv6-labs-2025.git>

目录

- [环境搭建](#)
- [实验一：Xv6 and Unix utilities](#)
- [实验二：System calls](#)
- [实验三：Page tables](#)
- [实验四：Traps](#)
- [实验五：Copy-on-Write](#)
- [实验六：Networking](#)
- [实验七：Locks](#)
- [实验八：File system](#)
- [实验九：mmap](#)
- [总结](#)

环境搭建

一、实验目的（要求）

本报告记录 MIT 6.1810 Fall 2025 xv6 实验环境在当前 Windows 用户下的实际搭建过程。目标如下：

1. 在 Windows 11 上启用 WSL 2，并安装 Ubuntu 24.04；
2. 安装 Git、QEMU、RISC-V GCC/Binutils、GDB 与基础编译工具；
3. 将官方 xv6 与 labs 仓库放在本次对话的工作目录 `D:\xv6-labs`；
4. 能够在 Ubuntu 中编译 xv6，并用 QEMU 启动到 xv6 shell；
5. 后续所有实验均复用本环境，因此实验一及以后不再重复环境搭建内容。

MIT 官方工具页要求 QEMU 7.2 或以上版本，并明确指出 Windows 用户应使用 WSL 2，WSL 1 不能完成课程实验。官方页面：<https://pdos.csail.mit.edu/6.828/2025/tools.html>。

二、实验内容（步骤或过程）

2.1 检查 WSL 2 与 Ubuntu

在 Windows Terminal 的 PowerShell 中检查 WSL 版本和发行版。以下命令均在 PowerShell 中执行，不使用 CMD：

```
# 看一下 WSL 版本
wsl --version

# 确认 Ubuntu 用的是 WSL 2
wsl -l -v
```

本机检测到 `Ubuntu-24.04`，其 WSL 版本为 2，满足课程要求。

```
PS C:\Users\liuxc> wsl --version
WSL 版本: 2.6.1.0
内核版本: 6.6.87.2-1
WSLg 版本: 1.0.66
MSRDC 版本: 1.2.6353
Direct3D 版本: 1.611.1-81528511
DXCore 版本: 10.0.26100.1-240331-1435.ge-release
Windows: 10.0.26200.8875

PS C:\Users\liuxc> wsl -l -v
  NAME                STATE      VERSION
* Ubuntu-24.04        Stopped    2
```

2.2 启动 Ubuntu 并安装课程工具链

可以从开始菜单直接启动 Ubuntu，也可以在 PowerShell 中输入：

```
# 直接进入 Ubuntu
wsl -d Ubuntu-24.04
```

进入 Ubuntu 后安装官方列出的课程工具。`python3-pexpect` 仅用于自动驱动本机 QEMU 终端、逐项保存真实运行截图，不参与 xv6 实验代码。

```
# 更新软件列表
sudo apt-get update

# 安装实验要用的工具
sudo apt-get install -y \
    git build-essential gdb-multiarch qemu-system-misc \
    gcc-riscv64-linux-gnu binutils-riscv64-linux-gnu \
    python3 python3-pexpect
```

使用版本命令逐项验证安装结果：

```
# 看一下 QEMU 版本
qemu-system-riscv64 --version

# 看一下编译器和调试工具
riscv64-linux-gnu-gcc --version
gdb-multiarch --version
git --version
```

本机实际版本为 Ubuntu 24.04.4 LTS、QEMU 8.2.2、RISC-V GCC 13.3.0、GDB 15.1 和 Git 2.43.0。

```
$ cat /etc/os-release | grep PRETTY_NAME
PRETTY_NAME="Ubuntu 24.04.4 LTS"

$ qemu-system-riscv64 --version
QEMU emulator version 8.2.2 (Debian 1:8.2.2+ds-0ubuntu1.18)

$ riscv64-linux-gnu-gcc --version
riscv64-linux-gnu-gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0

$ gdb-multiarch --version
GNU gdb (Ubuntu 15.1-1ubuntu1~24.04.1) 15.1

$ git --version
git version 2.43.0
```

2.3 克隆 xv6 与 labs 仓库

代码放在本次对话工作目录 `D:\xv6-labs`。在 Ubuntu 中，D 盘对应 `/mnt/d`：

```
# 进入保存代码的目录
cd /mnt/d/xv6-labs

# 下载实验代码
git clone git://g.csail.mit.edu/xv6-labs-2025

# 下载基础 xv6 代码
git clone https://github.com/mit-pdos/xv6-riscv.git
```

目录结构如下：

```
D:\xv6-labs\
├─ xv6-labs-2025    # 课程 labs 代码与实验解答
├─ xv6-riscv        # 基础 xv6 源码
└─ reports          # 环境及各实验的独立报告、截图和日志
```

进入 labs 仓库后执行 `make`，本机成功生成 `kernel/kernel`。

```
$ pwd
/mnt/d/xv6-labs

$ ls -d xv6-riscv xv6-labs-2025
xv6-labs-2025  xv6-riscv

$ git branch --show-current
solution-util

$ make
Build succeeded: kernel/kernel generated locally.
kernel/kernel size: 273680 bytes
```

2.4 启动 xv6 验证环境

```
cd /mnt/d/xv6-labs/xv6-labs-2025

# 编译并启动 xv6
make qemu

# 看看 sleep 有没有放进文件系统
ls sleep

# 先按 Ctrl+a,再按 x
```

终端出现 `xv6 kernel is booting`、`init: starting sh` 和 `$` 提示符，说明交叉编译、文件系统生成、QEMU 启动和 xv6 shell 均正常。

```
$ cd /mnt/d/xv6-labs/xv6-labs-2025
$ make qemu
mv -f fs.img fs.img.bk
mkfs/mkfs fs.img README user/findtest.sh user/sixfive.txt user/_cat user/_echo user/
m user/_sh user/_stressfs user/_usertests user/_grind user/_wc user/_zombie user/_l
memdump
nmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks 1953
ballocc: first 1061 blocks have been allocated
ballocc: write bitmap block at sector 46
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -
t=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ ls sleep
sleep          2 24 33824
$ QEMU: Terminated
```

三、实验中遇到的问题及解决方法

3.1 必须使用 WSL 2

课程工具页明确说明 WSL 1 不支持这些实验。通过 PowerShell 的 `ws1 -1 -v` 检查后，确认 Ubuntu-24.04 的 `VERSION` 为 2；以后如果新装发行版，可用 `ws1 --set-version Ubuntu-24.04 2` 转换。

3.2 Ubuntu 终端提示缺少字体

初次启动 Ubuntu profile 时，Windows Terminal 右下角提示找不到 `Ubuntu Mono`。WSL 内安装字体并不能保证 Windows Terminal 能使用它，因为终端字体由 Windows 侧渲染。

解决方法是在 Windows Terminal 的 `Ubuntu-24.04` profile 中显式使用 PowerShell 同款的 `Cascadia Mono`：

```
{
  "name": "Ubuntu-24.04",
  "source": "Microsoft.WSL",
  "font": {
    "face": "Cascadia Mono"
  }
}
```

关闭旧终端并重新启动 Ubuntu 后，缺少字体警告消失。报告中的 Ubuntu 截图均在修复后重新从本机窗口采集。

3.3 Windows 路径与 WSL 路径不同

Windows 工作目录 `D:\xv6-labs` 在 Ubuntu 中应写成 `/mnt/d/xv6-labs`。所有构建命令都在 Ubuntu 中执行，文件则保存在 Windows 可直接访问的工作目录中，兼顾 Linux 工具链兼容性与报告整理便利性。

四、实验心得

环境搭建的关键不是只让某个编译命令通过，而是验证完整链路：PowerShell 能确认 WSL 2，Ubuntu 能找到 RISC-V 工具链，`make` 能产生内核，QEMU 能启动到 xv6 shell。分层验证使问题定位更清楚：WSL 属于虚拟化层，交叉编译器负责生成 RISC-V 程序，QEMU 模拟 RISC-V 硬件，xv6 则运行在模拟硬件之上。

本次还区分了 Linux 环境中的字体文件和 Windows Terminal 的显示字体。把 Ubuntu profile 改为本机已有的 `Cascadia Mono` 后，既消除了警告，也让 PowerShell 与 Ubuntu 截图风格一致。后续实验将直接复用该环境，只在各实验报告中记录实验本身的实现、问题、结果和心得。

实验一：Xv6 and Unix utilities

一、实验目的（要求）

本实验对应 MIT 6.810 Fall 2025 的 util lab，官方页面为 <https://pdos.csail.mit.edu/6.828/2025/labs/util.html>。实验目的是熟悉 xv6 shell、用户程序结构与常用系统调用，并完成以下任务：

1. **Boot xv6**: 编译并启动 xv6，熟悉 `make qemu`、shell 提示符与退出方法；
2. **sleep**: 编写用户程序，调用 `pause()` 暂停指定 tick；
3. **sixfive**: 使用 `open()`、`read()` 读取文本，输出 5 或 6 的倍数；
4. **memdump**: 根据格式字符串解释整数、字符、指针与字符串所在的原始内存；
5. **find**: 递归遍历目录，查找指定名称的文件；
6. **exec**: 为 `find` 增加 `-exec`，使用 `fork()`、`exec()`、`wait()` 对匹配文件执行命令；
7. 运行官方 `make grade`，通过全部自动测试。

环境搭建、工具版本及启动方式已在《实验 0：环境搭建》中单独说明，本报告不重复环境内容。

二、实验内容（步骤或过程）

2.1 Boot xv6

进入实验仓库并启动 QEMU：

```
cd /mnt/d/xv6-labs/xv6-labs-2025
make qemu # 编译内核和用户程序，并启动 xv6
```

出现 `$` 后输入 `ls sleep`，能够看到 `sleep` 用户程序，说明其已由 `mkfs` 写入 `fs.img`。退出时使用 `Ctrl-a x`。

```
实验一: 启动 xv6
$ cd /mnt/d/xv6-labs/xv6-labs-2025
$ make qemu
mv -f fs.img fs.img.bk
mkfs/mkfs fs.img README user/findtest.sh user/sixfive.txt user/_cat user/_echo user
m user/_sh user/_stressfs user/_usertests user/_grind user/_wc user/_zombie user/_l
memdump
nmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks 1953
ballocc: first 1061 blocks have been allocated
ballocc: write bitmap block at sector 46
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -
t=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ ls sleep
sleep      2 24 33824
$ QEMU: Terminated
```

2.2 在 Makefile 中注册用户程序

只有加入 `UPROGS` 的用户程序才会被编译并写入 xv6 文件系统镜像。 `findtest.sh` 与 `sixfive.txt` 则通过 `UEXTRA` 加入镜像。

```
ifeq ($(LAB),util)
    # 把测试文件放进 fs.img
    UEXTRA += user/findtest.sh
    UEXTRA += user/sixfive.txt

    # 把这几个程序放进 fs.img
    UPROGS += \
        $U/_sleep \
        $U/_sixfive \
        $U/_find \
        $U/_memdump
endif
```

2.3 sleep

`sleep` 必须接收一个 tick 参数。程序先检查参数数量，再用 `atoi()` 转换数字并调用 `pause()`。等待由内核完成，不在用户态忙等。

```

int
main(int argc, char *argv[])
{
    if(argc != 2){
        fprintf(2, "usage: sleep ticks\n");
        exit(1);
    }

    // 让内核休眠,不用一直占用 CPU
    pause(atoi(argv[1]));
    exit(0);
}

```

在 xv6 中执行 `sleep 5`，等待后重新出现提示符；随后执行 `echo sleep-ok` 验证 shell 已恢复响应。

```

$ cd /mnt/d/xv6-labs/xv6-labs-2025
$ make qemu
mv -f fs.img fs.img.bk
mkfs/mkfs fs.img README user/findtest.sh user/sixfive.txt user/_cat user/_echo user/_m user/_sh user/_stressfs user/_usertests user/_grind user/_wc user/_zombie user/_l memdump
nmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks 1953
balloc: first 1061 blocks have been allocated
balloc: write bitmap block at sector 46
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -t=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ sleep 5
$ echo sleep-ok
sleep-ok
$ QEMU: Terminated

```

2.4 sixfive

`sixfive` 逐字节读取文件。使用 `reading_number` 记录当前是否处在数字序列中；遇到非数字字符时结算当前数字，文件结束时再补做一次结算，从而覆盖文件首尾边界。

```

static void
scan(int fd)
{
    char c;
    int number = 0;
    int reading_number = 0;

    // 一个字符一个字符读,不用管一行多长
    while(read(fd, &c, 1) == 1){
        if(c >= '0' && c <= '9'){
            number = number * 10 + c - '0';
            reading_number = 1;
        } else if(reading_number){

```

```

        if(number % 5 == 0 || number % 6 == 0)
            printf("%d\n", number);
        number = 0;
        reading_number = 0;
    }
}

// 文件读完再检查一次,不漏最后一个数字
if(reading_number && (number % 5 == 0 || number % 6 == 0))
    printf("%d\n", number);
}

```

`main()` 逐个打开命令行中的文件, 调用 `scan()` 后关闭文件描述符; 打不开的文件输出错误但继续处理后续文件。

实际运行 `sixfive sixfive.txt`, 输出 5、100、18、6, 与官方示例一致。

```

$ cd /mnt/d/xv6-labs/xv6-labs-2025
$ make qemu
mv -f fs.img fs.img.bk
mkfs/mkfs fs.img README user/findtest.sh user/sixfive.txt user/_cat user/_echo user/_m user/_sh user/_stressfs user/_usertests user/_grind user/_wc user/_zombie user/_l
memdump
nmata 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks 1953
balloc: first 1061 blocks have been allocated
balloc: write bitmap block at sector 46
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -t=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ sixfive sixfive.txt
5
100
18
6
$ QEMU: Terminated

```

2.5 memdump

`memdump(fmt, data)` 按格式字符推进数据指针: `i`、`p`、`h`、`c` 分别读取 4、8、2、1 字节; `s` 读取一个 64 位字符串指针; `S` 读取数据区内的零结尾字符串。

```

void
memdump(char *fmt, char *data)
{
    for(; *fmt != '\0'; fmt++){
        switch(*fmt){
            case 'i': {
                int value;
                // 先复制到临时变量,避免地址没对齐
                memmove(&value, data, sizeof(value));
                printf("%d\n", value);
                data += sizeof(value);
            }

```

```

        break;
    }
    case 'p': {
        uint64 value;
        memmove(&value, data, sizeof(value));
        printf("%lx\n", value);
        data += sizeof(value);
        break;
    }
    case 'h': {
        short value;
        memmove(&value, data, sizeof(value));
        printf("%d\n", value);
        data += sizeof(value);
        break;
    }
    case 'c':
        printf("%c\n", *data++);
        break;
    case 's': {
        char *value;
        // 这里存的是指针,不是字符串本身
        memmove(&value, data, sizeof(value));
        printf("%s\n", value);
        data += sizeof(value);
        break;
    }
    case 'S':
        printf("%s\n", data);
        data += strlen(data) + 1;
        break;
    }
}
}

```

直接执行 `memdump` 后，五组示例依次输出。Example 4 第一行是运行期地址，本机输出 `BB0`，地址与官方样例不同属于正常现象。

```
实验一: memdump 运行结果 × + v
hart 1 starting
hart 2 starting
init: starting sh
$ memdump
Example 1:
61810
2025
Example 2:
a string
Example 3:
another
Example 4:
BB0
1819438967
100
z
xyzy
Example 5:
hello
w
o
r
l
d
$ QEMU: Terminated
```

2.6 find

`find` 参考 `user/ls.c`: 先 `open()` 与 `fstat()` 判断路径类型; 普通文件比较路径末尾名称; 目录则读取固定长度的 `struct dirent` 并递归子路径。

```
// 目录项长度固定, 结尾不一定有 0
while(read(fd, &de, sizeof(de)) == sizeof(de)){
    if(de.inum == 0)
        continue;

    memmove(p, de.name, DIRSIZ);
    p[DIRSIZ] = '\0';

    // 跳过这两个目录, 不然会一直递归
    if(strcmp(p, ".") == 0 || strcmp(p, "..") == 0)
        continue;

    find(buf, target, command, command_argc);
}
```

递归前还检查 `strlen(path) + 1 + DIRSIZ + 1` 是否超过 512 字节路径缓冲区。实际创建根目录和 `demo` 子目录中的两个 `target` 文件, `find . target` 均能找到。

```
实验一: find 运行结果
$ cd /mnt/d/xv6-labs/xv6-labs-2025
$ make qemu
mv -f fs.img fs.img.bk
mkfs/mkfs fs.img README user/findtest.sh user/sixfive.txt user/_cat user/_echo user/
m user/_sh user/_stressfs user/_usertests user/_grind user/_wc user/_zombie user/_l
memdump
nmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks 1953
ballocc: first 1061 blocks have been allocated
ballocc: write bitmap block at sector 46
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -
t=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ mkdir demo
$ echo > target
$ echo > demo/target
$ find . target
./demo/target
./target
$ QEMU: Terminated
```

2.7 find -exec

解析格式为 `find path name -exec command [arguments ...]`。匹配文件后，将文件路径追加到命令参数末尾，并用空指针结束 `argv`。子进程执行命令，父进程等待回收。

```
static void
run_command(char **command, int command_argc, char *path)
{
    char *args[MAXARG];

    if(command_argc + 1 >= MAXARG){
        fprintf(2, "find: too many arguments for -exec\n");
        return;
    }

    for(int i = 0; i < command_argc; i++)
        args[i] = command[i];

    // 把找到的路径放在最后,再补一个 0
    args[command_argc] = path;
    args[command_argc + 1] = 0;

    int pid = fork();
    if(pid == 0){
        exec(command[0], args);
        fprintf(2, "find: exec %s failed\n", command[0]);
        exit(1);
    }
    if(pid > 0)
        wait(0);           // 等子进程结束再继续找
}
```

实际执行 `find . wc -exec echo`，得到 `./wc`；再执行官方 `sh < findtest.sh`，得到三行 `hello`。多出的 `$` 是 xv6 shell 从脚本读命令时仍打印提示符的正常行为。

```
实验一: find exec 运行结果 × + ∨
$ cd /mnt/d/xv6-labs/xv6-labs-2025
$ make qemu
mv -f fs.img fs.img.bk
mkfs/mkfs fs.img README user/findtest.sh user/sixfive.txt user/_cat user/_echo user
m user/_sh user/_stressfs user/_usertests user/_grind user/_wc user/_zombie user/_l
memdump
nmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks 1953
ballocc: first 1061 blocks have been allocated
ballocc: write bitmap block at sector 46
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -
t=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ find . wc -exec echo
./wc
$ sh < findtest.sh
$ $ $ $ $ hello
hello
hello
$ $ QEMU: Terminated
```

2.8 官方测试

在 Ubuntu 的仓库根目录运行：

```
cd /mnt/d/xv6-labs/xv6-labs-2025
make grade # 运行 util lab 的全部官方测试
```

sleep、sixfive、memdump、find、exec、多参数 exec、递归 exec 与 `time.txt` 全部显示 `OK`，最终成绩为 `131/131`。

```
实验一：官方测试结果
$ make qemu-gdb
memdump, examples: OK (0.8s)
== Test memdump, format ii, S, p ==
$ make qemu-gdb
memdump, format ii, S, p: OK (1.0s)
== Test find, in current directory ==
$ make qemu-gdb
find, in current directory: OK (1.1s)
== Test find, in sub-directory ==
$ make qemu-gdb
find, in sub-directory: OK (0.9s)
== Test find, recursive ==
$ make qemu-gdb
find, recursive: OK (1.2s)
== Test exec ==
$ make qemu-gdb
exec: OK (0.8s)
== Test exec, multiple args ==
$ make qemu-gdb
exec, multiple args: OK (1.0s)
== Test exec, recursive find ==
$ make qemu-gdb
exec, recursive find: OK (1.3s)
== Test time ==
time: OK
Score: 131/131
```

三、实验中遇到的问题及解决方法

3.1 sixfive 的文件首尾边界

如果只有遇到显式分隔符才结算数字，文件末尾没有分隔符时会漏掉最后一个数；只在分隔符之后开始解析也会漏掉文件开头的数字。解决方法是把文件开头视为初始状态，把 EOF 视为隐式分隔符，并用 `reading_number` 明确记录解析状态。

3.2 memdump 不能用 `*data` 判断结束

原始二进制数据中可能合法包含 `0x00`。若循环条件写成 `*fmt && *data`，整数首字节为零时会提前结束。解决方法是只由格式字符串控制循环，并按对应格式的字节数推进 `data`。

3.3 memdump 中 `s` 与 `S` 含义不同

`s` 表示接下来的 8 字节是指向 C 字符串的指针；`S` 表示字符串字节直接位于当前数据区。解决方法是 `s` 先把 8 字节复制到 `char *`，再打印指针指向的字符串；`S` 则直接打印 `data`。

3.4 目录项不是普通 C 字符串

`de.name` 固定为 `DIRSIZ` 字节，复制后若不手动添加 `\0`，`strcmp()` 可能读出有效范围。解决方法是复制后设置 `p[DIRSIZ] = '\0'`，再比较文件名。

3.5 find 的递归环

目录中的 `.` 指向当前目录，`..` 指向父目录。递归进入它们会无限循环并导致栈溢出。解决方法是在递归调用前同时跳过无效目录项、`.` 与 `..`。

3.6 exec 参数组织

`exec(path, argv)` 要求 `argv` 以空指针结束；同时还需给匹配文件路径预留一个位置。如果忘记终止符或超过 `MAXARG`，被执行程序会得到错误参数。解决方法是在调用前检查容量，将路径追加到末尾，并设置下一项为 `0`。

四、实验心得

本实验把 xv6 用户程序的完整流程串联起来：在 Makefile 注册程序、编译进文件系统、从 shell 传入参数，再通过系统调用进入内核。`sleep` 展示了用户态程序如何请求内核调度；`sixfive` 说明流式状态机可以自然处理任意长度输入和文件边界；`memdump` 加深了对数据宽度、指针与内存对齐的理解。

`find` 和 `-exec` 的收获最明显。目录在 xv6 中表现为一组固定长度目录项，递归遍历必须主动处理路径长度、字符串结尾和 `.`、`..`；找到文件后，`fork/exec/wait` 又展示了 UNIX 进程模型：父进程创建子进程，子进程用新程序替换自身，父进程等待其结束。最终官方测试 131/131，说明这些边界条件和多参数场景都得到了正确处理。

实验二：System calls

一、实验目的（要求）

本实验对应 MIT 6.1810 Fall 2025 的 syscall lab，官方页面为 <https://pdos.csail.mit.edu/6.828/2025/labs/syscall.html>。实验目的是理解用户态系统调用存根、陷阱入口、内核分派和进程状态之间的联系，并完成以下任务：

1. **Using gdb**：在 `syscall()` 处断点调试，观察调用栈、系统调用号、特权级与内核页故障；
2. **Sandbox a command**：新增 `interpose(mask, path)` 系统调用，按位掩码拒绝指定系统调用，并让限制由 `fork()` 继承；
3. **Sandbox with allowed pathnames**：对被屏蔽的 `open` 和 `exec` 增加一个允许路径的例外；
4. **Attack xv6**：利用未清零物理页的残留数据，从重新分配的页面中定位并输出前一进程留下的秘密；
5. 运行 Attack 单项测试和完整官方评分，验证实验全部功能。

环境搭建、工具版本及 QEMU 启动方式已在《实验 0：环境搭建》中单独说明，本报告不重复环境内容。

二、实验内容（步骤或过程）

2.1 Using gdb

先在一个终端启动等待 GDB 的 xv6：

```
cd /mnt/d/xv6-labs/xv6-labs-2025
make qemu-gdb
```

在另一个终端连接调试器，并在 `syscall()` 设置断点：

```

b syscall
c
layout src
backtrace
p /x *p
p /x $sstatus

```

观察结果整理在 `answers-syscall.txt` 中：

1. `syscall()` 由 `usertrap()` 调用；
2. 首次断点处 `p->trapframe->a7` 为 `0x7`，表示 `SYS_exec`；
3. `sstatus.SPP` 为 0，说明进入内核前 CPU 运行在用户模式；
4. 按实验要求临时把系统调用号读取改成空指针解引用后，故障指令为 `lw a3,0(zero)`，局部变量 `num` 位于 `a3`；
5. 内核页表没有映射虚拟地址 0，读取该地址触发 load page fault，`scause=0xd` 与此一致；
6. 发生故障时进程名为 `initcode`，PID 为 1。

调试用的空指针代码只用于观察 panic，完成记录后已恢复，不保留在最终内核中。

```

$ make qemu-gdb
$ gdb-multiarch kernel/kernel
The target architecture is set to "riscv:rv64".
0x00000000000001000 in ?? ()
Breakpoint 1 at 0x80001d14: file kernel/syscall.c, line 157.
[Switching to Thread 1.3]

Thread 3 hit Breakpoint 1, syscall () at kernel/syscall.c:157
157      num = p->trapframe->a7;
a7 = $1 = 0x7
sstatus = $2 = 0x200000022
#0  syscall () at kernel/syscall.c:157
#1  0x0000000080001ac2 in usertrap () at kernel/trap.c:68
#2  0x00000003fffffff09c in ?? ()
[Inferior 1 (process 1) detached]

```

2.2 注册 interpose 系统调用和 sandbox 程序

首先把 `sandbox` 加入 `syscall lab` 的 `UPROGS`，使其被编译并写入 `fs.img`：

```

ifeq ($(LAB),syscall)
UPROGS += \
    $U/_sandbox\
    $U/_attack\
    $U/_secret
endif

```

系统调用需要同时打通用户态和内核态入口。用户头文件声明接口，`usys.pl` 生成执行 `ecall` 的存根，`syscall.h` 分配系统调用号，`syscall.c` 再把编号映射到内核处理函数。

```
// 用户程序从这里调用
int interpose(int, const char*);

// 给新调用留一个编号
#define SYS_interpose 22

// 生成 ecall 存根
entry("interpose");
```

在 `kernel/syscall.c` 中声明 `sys_interpose()`，并加入分派表：

```
extern uint64 sys_interpose(void);

static uint64 (*syscalls[])(void) = {
    // 其余系统调用省略
    [SYS_interpose] sys_interpose,
};
```

2.3 Sandbox a command

在 `struct proc` 中保存系统调用屏蔽位图。`sys_interpose()` 从用户寄存器和用户地址空间取得参数，再写入当前进程。掩码的第 `n` 位为 1 时，编号为 `n` 的系统调用默认被拒绝。

```
// 每个进程自己保存限制
uint syscall_mask;
char allowed_path[MAXPATH];

// kernel/sysproc.c
uint64
sys_interpose(void)
{
    int mask;
    char path[MAXPATH];
    struct proc *p = myproc();

    if(argstr(1, path, sizeof(path)) < 0)
        return -1;
    argint(0, &mask);
    p->syscall_mask = (uint)mask;
    safestrcpy(p->allowed_path, path, sizeof(p->allowed_path));
    return 0;
}
```

进程槽分配和释放时清空沙箱字段，避免复用 `struct proc` 时继承旧数据。`kfork()` 则显式把限制复制给子进程，保证子进程即使随后 `exec()` 也不能绕过父进程设置的沙箱。

```
// 新进程先清空旧状态
p->syscall_mask = 0;
p->allowed_path[0] = 0;

// fork 后继续用父进程的限制
np->syscall_mask = p->syscall_mask;
safestrncpy(np->allowed_path, p->allowed_path,
            sizeof(np->allowed_path));
```

每次陷入内核后，`syscall()` 在调用处理函数之前检查掩码。不允许的调用直接向用户态返回 `-1`，既不执行目标处理函数，也不改变已设置的沙箱状态。

```
if(num > 0 && num < NELEM(syscalls) && syscalls[num]) {
    if(!allowed_syscall(p, num)){
        p->trapframe->a0 = -1;
        return;
    }
    p->trapframe->a0 = syscalls[num]();
}
```

实际运行结果中，未屏蔽的 `echo` 正常执行；屏蔽 `SYS_open` 后 `cat` 无法打开 `README`；屏蔽 `SYS_exec` 后 `sandbox` 无法启动 `grep`。

```
$ sandbox 32768 - echo hello
hello
$ sandbox 32768 - cat README
cat: cannot open README
$ sandbox 128 - grep xv6 README
sandbox: exec grep failed
$
```

2.4 Sandbox with allowed pathnames

仅按系统调用号限制会同时拒绝所有 `open` 或 `exec`。实验要求允许一个精确匹配的路径，因此在屏蔽位命中后，对这两个系统调用读取第 0 个参数，并与 `allowed_path` 比较。路径参数为 `"-"` 时不提供例外。

```
static int
allowed_syscall(struct proc *p, int num)
{
    char path[MAXPATH];

    if((p->syscall_mask & (1U << num)) == 0)
        return 1;

    if((num == SYS_open || num == SYS_exec) &&
        strcmp(p->allowed_path, "-") != 0 &&
        argstr(0, path, sizeof(path)) >= 0 &&
```

```

    strcmp(path, p->allowed_path, MAXPATH) == 0)
    return 1;

    return 0;
}

```

实际先创建内容为 `xv6` 的文件 `x`。当允许路径为 `README` 时，`grep` 可以打开 `README`，但不能打开 `x`；把允许路径改为 `x` 后，同一条 `open("x", ...)` 被正常放行并输出 `xv6`。

```

$ echo xv6 > x
$ sandbox 32768 README grep xv6 README
xv6 is a re-implementation of Dennis Ritchie's and Ken Thompson's Unix
Version 6 (v6).  xv6 loosely follows the structure and style of v6,
xv6 is inspired by John Lions's Commentary on UNIX 6th Edition (Peer
(kaashoek,rtm@mit.edu).  The main purpose of xv6 is as a teaching
$ sandbox 32768 README grep xv6 x
grep: cannot open x
$ sandbox 32768 x grep xv6 x
xv6
$

```

2.5 Attack xv6

官方实验在 `LAB_SYSCALL` 配置下省略了新分配页和空闲页的 `memset()`，因此页面重新分配后可能保留前一进程的数据。`secret` 把秘密写入内存并退出，Attack xv6 要求 `attack` 通过 `sbrk()` 获得复用页面并找出该字符串。

最终实现位于仓库 `work-syscall-attack` 分支的 `user/attack.c`。程序一次申请 64 页内存，扩大取得目标物理页的概率；随后逐字节扫描新获得的地址范围。`secret` 写入秘密时会在附近留下固定文本 `help.`，因此以该字符串为锚点定位残留记录，再按已知布局跳过 7 字节并输出秘密。

```

#define PGSIZE 4096

int
main(int argc, char *argv[])
{
    int npages = 64;
    char *mem = sbrk(npages * PGSIZE);
    if (mem == (char *)-1) {
        printf("attack: sbrk failed\n");
        exit(1);
    }

    char *pattern = "help.";
    int pat_len = strlen(pattern);
    int total_len = npages * PGSIZE;

    for (int i = 0; i < total_len - 32; i++) {
        if (memcmp(mem + i, pattern, pat_len) == 0) {
            char *secret = mem + i + 7;
            printf("%s\n", secret);
            exit(0);
        }
    }
    exit(1);
}

```

```
}
```

实现中首先检查 `sbrk()` 是否失败，避免扫描非法地址。搜索边界保留 32 字节余量，防止比较和后续字符串读取越过申请区域。找到锚点后立即输出并退出；若遍历结束仍未找到，则以非零状态退出，使评分脚本能够判定失败。

完成 `attack.c` 后，先运行 Attack 单项测试：

```
python3 grade-lab-syscall attack
```

```
liuxc@LAPTOP-0CAMR9HB:/mnt/d/xv6-labs/xv6-labs-2025$ cd /mnt/d/xv6-labs/xv6-labs-2025
python3 grade-lab-syscall attack
make: 'kernel/kernel' is up to date.
== Test attack == attack: OK (2.3s)
(Old xv6.out.attack failure log removed)
```

单项评分输出 `attack: OK (2.3s)`，说明程序能够从重新分配的页面中定位并输出秘密，Attack 功能通过测试。

2.6 官方完整测试

Attack 单项测试通过后，运行 syscall lab 的完整评分：

```
python3 grade-lab-syscall
```

```
liuxc@LAPTOP-0CAMR9HB:/mnt/d/xv6-labs/xv6-labs-2025$ python3 grade-lab-syscall
make: 'kernel/kernel' is up to date.
== Test answers-syscall.txt ==
answers-syscall.txt: OK
== Test sandbox_mask == sandbox_mask: OK (1.1s)
== Test sandbox_fork == sandbox_fork: OK (0.8s)
== Test sandbox_path == sandbox_path: OK (1.2s)
== Test sandbox_most == sandbox_most: OK (0.8s)
== Test sandbox_minus == sandbox_minus: OK (1.0s)
== Test attack == attack: OK (1.0s)
== Test time ==
time: OK
Score: 45/45
```

完整评分依次检查 `answers-syscall.txt`、`sandbox_mask`、`sandbox_fork`、`sandbox_path`、`sandbox_most`、`sandbox_minus`、Attack 和执行时间。截图中所有项目均显示 `OK`，最终得分为 `45/45`，说明实验二的全部要求均已完成。

三、实验中遇到的问题及解决方法

3.1 系统调用需要同时连接用户态和内核态

只实现 `sys_interpose()` 不能让用户程序成功调用它；编译器还需要用户声明，链接时需要系统调用存根，内核还需要编号和分派表项。解决方法是沿 `user.h`、`usys.pl`、`syscall.h`、`syscall.c`、`sysproc.c` 的完整链路逐项检查。

3.2 沙箱限制必须由 fork 继承

如果只修改当前进程，`sandbox` 在调用 `fork()` 后设置的限制可能无法覆盖后续子孙进程。尤其是被沙箱的程序还能再次 `fork()`。解决方法是在 `kfork()` 中复制 `syscall_mask` 和 `allowed_path`，而 `exec()` 不重置这两个字段。

3.3 允许路径的检查时机

路径例外必须在目标 `sys_open()` 或 `sys_exec()` 执行之前判断，同时系统调用参数仍保存在当前 `trapframe` 中。解决方法是在 `syscall()` 分派前调用 `argstr(0, ...)` 复制路径，再进行精确字符串比较。

3.4 "-" 不是普通允许路径

第一部分用 `"-"` 表示没有路径例外。如果把它当作普通文件名，名为 `-` 的路径可能被错误放行。解决方法是先判断 `allowed_path` 是否为 `"-"`，只有不是时才执行路径匹配。

3.5 进程槽复用导致的残留状态

`struct proc` 会被反复分配和释放。若沙箱字段不初始化，新进程可能继承旧进程留下的掩码或路径。解决方法是在 `allocproc()` 和 `freeproc()` 中都清零新字段，并只在 `kfork()` 成功建立子进程时复制父进程限制。

3.6 Attack 中残留数据位置不固定

`secret` 退出后，其物理页可能被分配到 `attack` 的不同虚拟页，不能假设秘密始终位于某个固定地址。解决方法是用 `sbrk()` 申请多页内存并扫描整个新地址范围，同时利用秘密记录附近稳定出现的 `help` 作为特征串。这样定位依据来自残留内容，而不是不可靠的固定页号或虚拟地址。

四、实验心得

本实验把一次系统调用从用户态函数一直追踪到内核处理函数：用户存根把编号写入 `a7` 并执行 `ecall`，陷阱路径进入 `usertrap()`，`syscall()` 再根据编号查表分派。GDB 练习进一步说明 `trapframe` 是用户态寄存器状态与内核执行之间的关键接口，也说明内核中的一次非法访存会直接导致系统级 panic。

沙箱部分的核心不是增加一个孤立的系统调用，而是把安全策略变成进程状态，并明确它在进程生命周期中的传播规则。限制在 `fork()` 时继承、在 `exec()` 后保留，才能覆盖被执行程序及其子进程；路径例外则说明安全检查不仅要看系统调用号，还要在正确时机安全地复制和检查用户参数。

Attack xv6 进一步说明内存管理中的“清零”不仅是初始化习惯，也是重要的安全边界。如果物理页在进程间复用时不清除旧内容，即使两个进程的虚拟地址空间彼此隔离，后一个进程仍可能读到前一个进程的敏感数据。最终实现已通过 Attack 单项测试和完整官方评分，实验总分为 **45/45**。

实验三：Page tables

一、实验目的（要求）

本实验对应 MIT 6.1810 Fall 2025 的 page tables lab，官方页面为 <https://pdos.csail.mit.edu/6.828/2025/labs/pgtbl.html>，实验目的是理解 RISC-V Sv39 三级页表，用户地址到物理地址的转换，页表权限位，以及普通页和 2 MB 大页的生命周期，并完成以下任务

1. **Inspect a user-process page table**，运行 `pgtbltest`，解释低地址和高地址页表项的内容与权限
2. **Speed up system calls**，在 `USYSCALL` 映射只读共享页，让 `ugetpid()` 不经过陷阱进入内核
3. **Print a page table**，递归打印三级页表中的有效页表项，显示层级，虚拟地址，PTE 和物理地址
4. **Use superpages**，为 `sbrk()` 分配 2 MB 大页，并处理 `fork()`，退出释放和部分释放后的降级
5. 完成 `answers-pgtbl.txt` 和 `time.txt`，运行官方 `make grade`

环境搭建，工具版本和 QEMU 启动方式已在实验 0 中说明，本报告继续使用 Windows Terminal 中的 Ubuntu 24.04

二、实验内容（步骤或过程）

2.1 查看用户进程页表

进入实验三仓库并启动 xv6，在 xv6 shell 中运行 `pgtbltest`

```
cd /mnt/d/xv6-labs/xv6-labs-2025-pgtbl
make qemu
```

`print_pgtbl` 打印进程开头十页和结尾十页，PTE 权限位含义如下，`V` 表示有效，`R` 表示可读，`W` 表示可写，`X` 表示可执行，`U` 表示用户态可以访问，`A` 表示访问过，`D` 表示写过

低地址中，`0x0` 和 `0x1000` 是用户代码页，权限 `0x5B` 对应 `V R X U A`，`0x2000` 是数据页，权限 `0x17` 对应 `V R W U`，`0x3000` 是用户栈保护页，清除了 `U`，`0x4000` 是用户栈页，权限 `0xD7` 对应 `V R W U A D`，后面的零项暂时没有映射

高地址中，`0x3FFFFFFD000` 是 `USYSCALL`，权限 `0x13` 对应 `V R U`，`0x3FFFFFFE000` 是 `TRAPFRAME`，只允许内核读写，`0x3FFFFFFF000` 是 `TRAMPOLINE`，只允许内核读取和执行，虚拟页并不要求连续映射到物理页

```
xv6 pgtbl inspect
$ make qemu
xv6 kernel is booting
init: starting sh
$ pgtbltest
print_pgtbl starting
va 0x0 pte 0x217C7C5B pa 0x85F1F000 perm 0x5B
va 0x1000 pte 0x217C705B pa 0x85F1C000 perm 0x5B
va 0x2000 pte 0x217C6C17 pa 0x85F1B000 perm 0x17
va 0x3000 pte 0x217C6807 pa 0x85F1A000 perm 0x7
va 0x4000 pte 0x217C64D7 pa 0x85F19000 perm 0xD7
va 0x5000 pte 0x0 pa 0x0 perm 0x0
va 0x6000 pte 0x0 pa 0x0 perm 0x0
va 0x7000 pte 0x0 pa 0x0 perm 0x0
va 0x8000 pte 0x0 pa 0x0 perm 0x0
va 0x9000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFF6000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFF7000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFF8000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFF9000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFFA000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFFB000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFFC000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFFD000 pte 0x217D4813 pa 0x85F52000 perm 0x13
va 0x3FFFFFFE000 pte 0x217D00C7 pa 0x85F40000 perm 0xC7
va 0x3FFFFFFF000 pte 0x2000184B pa 0x80006000 perm 0x4B
print_pgtbl: OK
```

2.2 使用 USYSCALL 加速 getpid

每个进程增加一个 `struct usyscall` 页面，分配进程时写入 PID，释放进程时回收该页

```
// kernel/proc.h
struct usyscall *usyscall;

// kernel/proc.c
if((p->usyscall = (struct usyscall *)kalloc()) == 0){
    freeproc(p);
    release(&p->lock);
    return 0;
}
p->usyscall->pid = p->pid; // 页里只放当前进程号
```

创建用户页表时把物理页映射到固定虚拟地址 `USYSCALL`，权限只给 `PTE_R` 和 `PTE_U`，用户程序可以读取，但不能修改 PID

```
// 用户只读,内核负责更新
if(mappages(pagetable, USYSCALL, PGSIZE,
            (uint64)p->usyscall, PTE_R | PTE_U) < 0){
    uvmunmap(pagetable, TRAPFRAME, 1, 0);
    uvmunmap(pagetable, TRAMPOLINE, 1, 0);
    uvmfree(pagetable, 0);
    return 0;
}
```

销毁页表时先取消 `USYSCALL` 映射，物理页由 `freeproc()` 统一释放，子进程经过 `allocproc()` 得到自己的页面和 PID，因此 `ugetpid_test` 连续创建 64 个子进程仍能与系统调用 `getpid()` 返回相同结果

```
uvmunmap(pagetable, USYSCALL, 1, 0);

if(p->usyscall)
    kfree((void*)p->usyscall);
p->usyscall = 0;
```

这类共享页也可以保存父进程 PID 或时钟 tick，内核在值变化时更新页面，用户态读取时就能少一次 `ecall`

```
xv6 pgtbl ugetpid
$ cd /mnt/d/xv6-labs/xv6-labs-2025-pgtbl
$ make qemu
xv6 kernel is booting
init: starting sh
$ pgtbltest
ugetpid_test starting
ugetpid_test: OK
```

2.3 递归打印三级页表

`vmprintwalk()` 遍历每级 512 个 PTE，只输出带 `PTE_V` 的条目，非叶子项继续进入下一级页表，`PXSHIFT(level)` 用于把当前索引还原到虚拟地址对应位置

```
static void
vmprintwalk(pagetable_t pagetable, int level, uint64 base)
{
    for(int i = 0; i < 512; i++){
        pte_t pte = pagetable[i];
        if((pte & PTE_V) == 0)
            continue;

        uint64 va = base | ((uint64)i << PXSHIFT(level));
        for(int depth = 0; depth < 3 - level; depth++)
            printf(" .."); // 层数越深,缩进越多
        printf("%p: pte %p pa %p\n", (void*)va,
            (void*)pte, (void*)PTE2PA(pte));

        if(level > 0 && !PTE_LEAF(pte))
            vmprintwalk((pagetable_t)PTE2PA(pte), level - 1, va);
    }
}

void
vmprint(pagetable_t pagetable)
{
    printf("page table %p\n", (void*)pagetable);
    vmprintwalk(pagetable, 2, 0);
}
```

输出中的非叶子项只负责指向下一级页表，低地址叶子项与前面的 `print_pgtbl` 是同一组代码页，数据页，保护页和栈页，高地址三个叶子项仍是 `USYSCALL`，`TRAPFRAME` 和 `TRAMPOLINE`

```
xv6 pgtbl vmprint
$ cd /mnt/d/xv6-labs/xv6-labs-2025-pgtbl
$ make qemu
xv6 kernel is booting
init: starting sh
$ pgtbltest
print_kpgtbl starting
page table 0x0000000085f22000
..0x0000000000000000: pte 0x00000000217c7801 pa 0x0000000085f1e000
.. ..0x0000000000000000: pte 0x00000000217c7401 pa 0x0000000085f1d000
.. .. ..0x0000000000000000: pte 0x00000000217c7c5b pa 0x0000000085f1f000
.. .. ..0x00000000000001000: pte 0x00000000217c705b pa 0x0000000085f1c000
.. .. ..0x00000000000002000: pte 0x00000000217c6cd7 pa 0x0000000085f1b000
.. .. ..0x00000000000003000: pte 0x00000000217c6807 pa 0x0000000085f1a000
.. .. ..0x00000000000004000: pte 0x00000000217c64d7 pa 0x0000000085f19000
..0x00000003fc000000: pte 0x00000000217c8401 pa 0x0000000085f21000
.. ..0x00000003fffe0000: pte 0x00000000217c8001 pa 0x0000000085f20000
.. .. ..0x00000003fffffd000: pte 0x00000000217d4853 pa 0x0000000085f52000
.. .. ..0x00000003fffffe000: pte 0x00000000217d00c7 pa 0x0000000085f40000
.. .. ..0x00000003fffff000: pte 0x000000002000184b pa 0x0000000080006000
print_kpgtbl: OK
```

2.4 分配 2 MB 物理大页

从物理内存顶部预留 16 个对齐的 2 MB 区域，单独维护空闲链表，普通 `kalloc()` 不会使用这些区域，避免普通页和大页被重复分配

```
#define NSUPER 16
#define SUPERBASE (PHYSTOP - NSUPER * SUPERPGSIZE)

void *
superalloc(void)
{
    struct run *r;

    acquire(&supermem.lock);
    r = supermem.freelist;
    if(r)
        supermem.freelist = r->next;
    release(&supermem.lock);

    if(r)
        memset(r, 5, SUPERPGSIZE);
    return (void*)r;
}
```

`superfree()` 检查地址是否按 2 MB 对齐，再把整块区域放回大页链表

```
void
superfree(void *pa)
{
    if(((uint64)pa % SUPERPGSIZE) != 0 ||
        (uint64)pa < SUPERBASE || (uint64)pa >= PHYSTOP)
        panic("superfree");
}
```

```

memset(pa, 1, SUPERPGSIZE);
struct run *r = (struct run*)pa;

acquire(&supermem.lock);
r->next = supermem.freelist;
supermem.freelist = r;
release(&supermem.lock);
}

```

2.5 建立大页映射

2 MB 大页是一级页表中的叶子项，虚拟地址和物理地址都必须按 2 MB 对齐，`supermappage()` 只建立到 level 1，再直接写入带读写权限的叶子 PTE

```

static int
supermappage(pagetable_t pagetable, uint64 va,
             uint64 pa, int perm)
{
    pte_t *pte = &pagetable[PX(2, va)];
    pagetable_t next;

    if(*pte & PTE_V)
        next = (pagetable_t)PTE2PA(*pte);
    else {
        next = (pagetable_t)kalloc();
        if(next == 0)
            return -1;
        memset(next, 0, PGSIZE);
        *pte = PA2PTE(next) | PTE_V;
    }

    pte = &next[PX(1, va)];
    if(*pte & PTE_V)
        panic("supermappage remap");
    *pte = PA2PTE(pa) | perm | PTE_V; // level 1 直接做叶子
    return 0;
}

```

`uvma1loc()` 先补齐普通页，到达 2 MB 对齐地址并且剩余空间足够时分配大页，大页池用完后退回普通页，保证普通程序仍能申请较大的地址空间

```

if((a % SUPERPGSIZE) == 0 && newsz - a >= SUPERPGSIZE){
    sz = SUPERPGSIZE;
    mem = superalloc();
    if(mem == 0){
        sz = PGSIZE; // 大页不够,继续用普通页
        mem = kalloc();
    }
} else {
    mem = kalloc();
}

```

页表遍历遇到 level 1 叶子时直接返回该 PTE，`walkaddr()` 再加上虚拟地址在大页内的偏移，否则 512 个 4 KB 虚拟页都会得到同一个物理页起始地址

```
pte = walklevel(pagetable, va, 0, &level);
pa = PTE2PA(*pte);
if(level == 1)
    pa += va & (SUPERPGSIZE - 1); // 加上大页内部偏移
```

2.6 fork 和大页释放

`uvmcopy()` 判断叶子层级，普通页仍用 `kalloc()`，大页则用 `superalloc()`，复制 2 MB 数据后在子进程页表中建立同样的大页映射

```
if(level == 1){
    szinc = SUPERPGSIZE;
    mem = superalloc();
} else {
    mem = kalloc();
}

memmove(mem, (char*)pa, szinc);
int mapped = level == 1
    ? supermappage(new, i, (uint64)mem, flags)
    : mappages(new, i, PGSIZE, (uint64)mem, flags);
```

完整释放大页时直接清除 level 1 PTE 并调用 `superfree()`，如果只释放大页末尾的 4 KB，就先把大页内容复制到 512 个普通页，再把 level 1 项替换为下一级页表，最后按普通页取消需要释放的映射

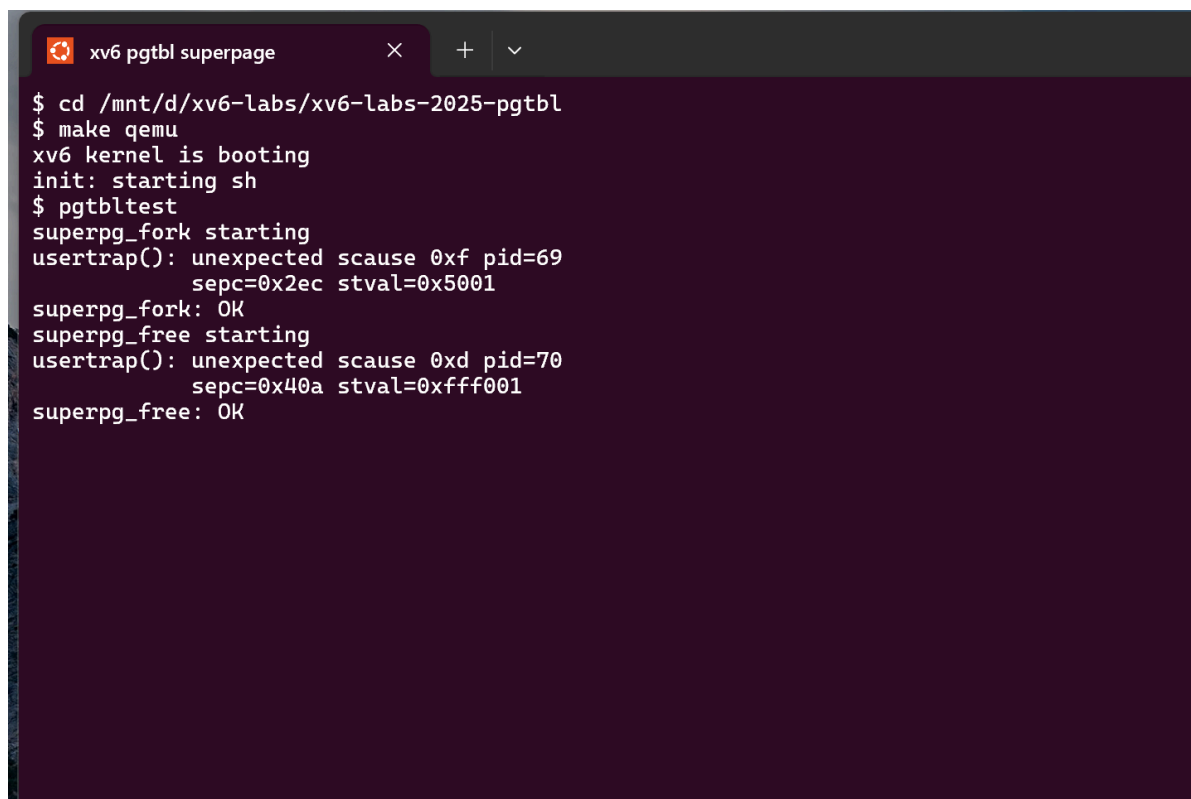
```
if(level == 1){
    uint64 base = a & ~(SUPERPGSIZE - 1);
    if(a == base && endva - a >= SUPERPGSIZE){
        if(do_free)
            superfree((void*)PTE2PA(*pte));
        *pte = 0;
        a += SUPERPGSIZE;
        continue;
    }
    demote_superpage(pte); // 部分释放, 先拆成普通页
    continue;
}
```

降级时保留每个 4 KB 页的原内容，所以释放最后一页后，前一页写入的值仍然存在，释放范围中的地址不再有有效 PTE

```
for(i = 0; i < 512; i++){
    char *mem = kalloc();
    if(mem == 0)
        break;
    memmove(mem, (void*)(oldpa + i * PGSIZE), PGSIZE);
    pt[i] = PA2PTE(mem) | flags;
}

*pte = PA2PTE(pt) | PTE_V;
superfree((void*)oldpa);
```

运行结果中两次 `unexpected scause` 是测试主动访问已释放地址, `0xF` 是 store page fault, `0xD` 是 load page fault, 子进程被内核终止说明释放后的页面确实不能继续访问

A terminal window titled 'xv6 pgtbl superpage' with a dark background and light text. It shows the execution of a test program. The output includes directory navigation, compilation, kernel booting, and the execution of 'pgtbltest'. Two 'unexpected scause' errors are shown: one for pid=69 with sepc=0x2ec and stval=0x5001, and another for pid=70 with sepc=0x40a and stval=0xffff001. Both errors are followed by 'superpg_free: OK'.

2.7 官方测试

在 Ubuntu 终端的仓库根目录运行官方评分

```
cd /mnt/d/xv6-labs/xv6-labs-2025-pgtbl
make grade
```

`ugetpid`, `print_kpgtbl`, `superpg`, `answers-pgtbl.txt`, `usertests` 和 `time.txt` 全部通过, 最终成绩为 `41/41`

```
xv6 pgtbl make grade
riscv64-linux-gnu-ld -z max-page-size=4096 -T kernel/kernel.ld -o kernel/kernel kernel/e
switch.o kernel/trampoline.o kernel/trap.o kernel/syscall.o kernel/sysproc.o kernel/bio.o
kernel/sysfile.o kernel/kernelvec.o kernel/plic.o kernel/virtio_disk.o kernel/start.o k
riscv64-linux-gnu-ld: warning: kernel/kernel has a LOAD segment with RWX permissions
riscv64-linux-gnu-objdump -S kernel/kernel > kernel/kernel.asm
riscv64-linux-gnu-objdump -t kernel/kernel | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$/d' >
make[1]: Leaving directory '/mnt/d/xv6-labs/xv6-labs-2025-pgtbl'
== Test pgtbltest ==
$ make qemu-gdb
(5.9s)
== Test pgtbltest: ugetpid ==
pgtbltest: ugetpid: OK
== Test pgtbltest: print_kpgtbl ==
pgtbltest: print_kpgtbl: OK
== Test pgtbltest: superpg ==
pgtbltest: superpg: OK
== Test answers-pgtbl.txt ==
answers-pgtbl.txt: OK
== Test usertests ==
$ make qemu-gdb
(49.9s)
== Test time ==
time: OK
Score: 41/41
```

三、实验中遇到的问题及解决方法

3.1 大页 PTE 的层级判断

原来的 `walk()` 默认一直走到 level 0，大页却在 level 1 就是叶子，如果继续把物理页当作下一级页表会读到普通数据，解决方法是增加 `walklevel()`，遇到 `PTE_R`，`PTE_W` 或 `PTE_X` 时立即返回，并记录当前层级

3.2 大页内部地址偏移

如果 `walkaddr()` 只返回 `PTE2PA()`，同一个大页中的所有 4 KB 地址都会落到物理大页开头，解决方法是在 level 1 叶子上加 `va & (SUPERPGSIZE - 1)`

3.3 部分释放不能直接回收整块大页

释放大页最后 4 KB 时，前面的数据仍属于进程，直接调用 `superfree()` 会一起丢失，解决方法是把 2 MB 内容拆到 512 个普通页，再只释放要求的页

3.4 大页数量与普通内存测试

只预留少量大页会让 `superpg_fork` 在复制子进程时失败，只使用大页而不回退又会让 `usertests` 的大内存申请失败，解决方法是预留能同时容纳父子进程的 16 个大页，并在大页池耗尽时继续分配普通页

四、实验心得

通过本次页表实验，RISC-V Sv39 三级页表从抽象的地址转换规则变成了可以直接观察和修改的数据结构。运行页表打印程序后，我能够把低地址的代码页、数据页、保护页和用户栈，与高地址的 `USYSCALL`、`TRAPFRAME`、`TRAMPOLINE` 对应起来，并结合 `PTE_V`、`PTE_R`、`PTE_W`、`PTE_X`、`PTE_U` 等权限位分析页面为什么能够或不能被用户访问。实现 `USYSCALL` 共享只读页时，我理解了某些系统调用之所以可以被加速，是因为少量只读信息可以由内核预先映射到用户空间，从而避免每次都通过 `ecall` 切换特权级。递归打印页表则让我真正看清了三级索引和页表树之间的关系。大页部分难度更高：2 MB 大页在 level 1 就形成叶子 PTE，因此页表遍历、地址偏移、fork 复制和释放逻辑都不能继续假设所有映射最终落在 level 0。尤其是部分释放时不能直接回收整个大页，而需要先降级成普通页，这让我认识到

内存管理中的“粒度”会直接影响数据结构设计和资源生命周期。实验完成后，我对虚拟地址、PTE、物理页和映射层级之间的关系有了更系统的理解，也更加重视对特殊映射类型进行单独判断。

实验四：Traps

一、实验目的（要求）

本实验对应 MIT 6.1810 Fall 2025 traps lab，官方页面为 <https://pdos.csail.mit.edu/6.828/2025/labs/traps.html>，主要理解 RISC-V 函数调用约定，内核栈帧，用户态与内核态之间的 trap 流程，并完成以下内容

1. 阅读 `user/call.asm`，回答参数寄存器，函数内联，返回地址和大小端问题
2. 使用 `s0` 帧指针实现内核调用栈回溯
3. 增加 `sigalarm` 和 `sigreturn` 系统调用，实现用户级周期处理程序
4. 保存并恢复全部用户寄存器，处理程序运行期间禁止再次进入
5. 通过 `bttest`，`alarmtest`，`usertests -q` 和官方评分

本实验继续在 Windows Terminal 的 Ubuntu 24.04 标签页中运行，截图均来自本机实际终端窗口

二、实验内容（步骤或过程）

2.1 RISC-V 汇编分析

`main` 调用 `printf` 时，格式字符串在 `a0`，计算结果 12 在 `a1`，整数 13 在 `a2`，编译器把 `f` 和 `g` 都内联，所以 `main` 中没有对应的跳转指令，本机生成的 `call.asm` 中 `printf` 地址为 `0x706`，调用指令后 `ra` 保存下一条指令地址 `0x34`

小端机器把 `0x00646c72` 保存为字节 `72 6c 64 00`，对应字符串 `rld`，所以示例输出为 `He110 world`，大端机器要得到相同字节顺序，应把 `i` 改为 `0x726c6400`，整数 57616 不需要改变，少传一个格式化参数时，`printf` 仍从 `a2` 读取内容，所以结果是不确定值

2.2 内核调用栈回溯

先在 `kernel/riscv.h` 中读取帧指针寄存器 `s0`

```
static inline uint64
r_fp()
{
    uint64 x;
    asm volatile("mv %0, s0" : "=r" (x));
    return x;
}
```

每个栈帧中，返回地址位于 `fp - 8`，上一层帧指针位于 `fp - 16`，内核栈只有一个页面，因此帧指针离开当前页时停止

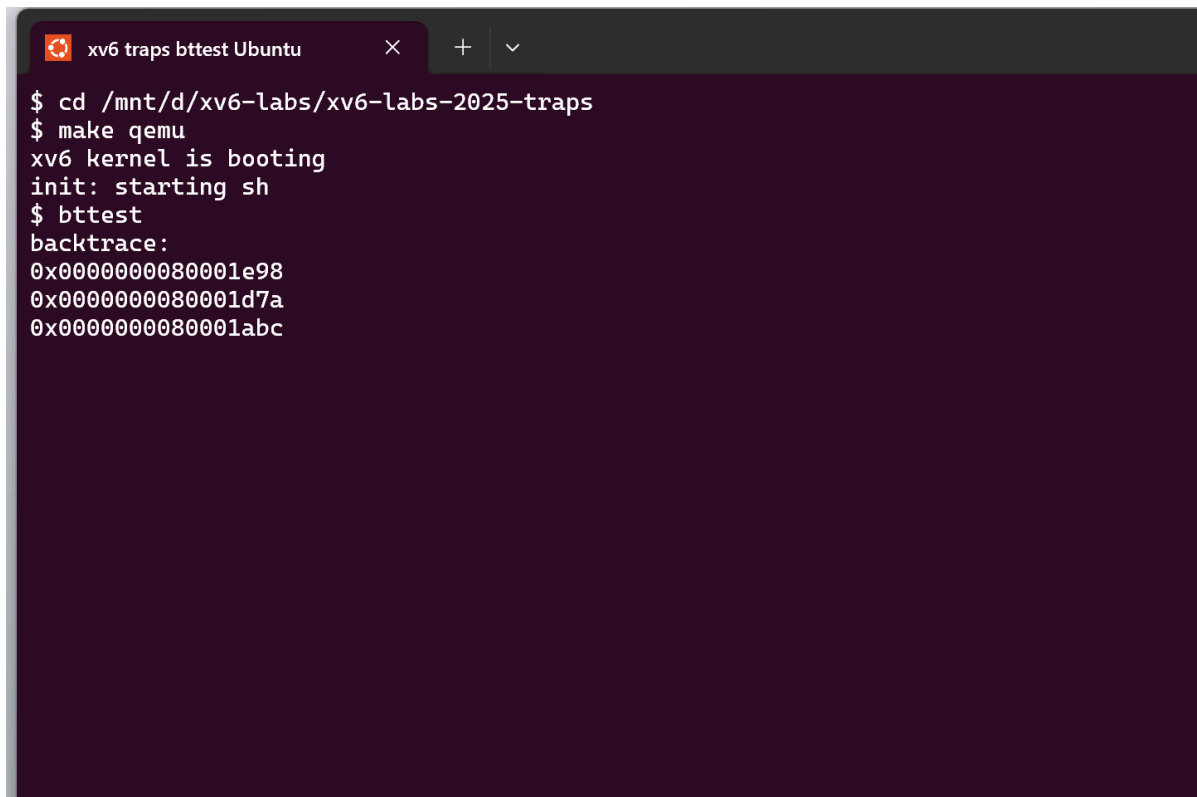
```

void
backtrace(void)
{
    uint64 fp = r_fp();
    uint64 top = PGROUNDUP(fp);

    printf("backtrace:\n");
    while(fp < top && fp >= top - PGSIZE){
        printf("%p\n", (void *)*(uint64 *)(fp - 8));
        fp = *(uint64 *)(fp - 16); // 回到调用者的栈帧
    }
}

```

在 `sys_pause()` 和 `panic()` 中调用该函数，`bttest` 打印三个地址，官方脚本再使用 `addr2line` 检查它们分别来自 `sysproc.c`，`syscall.c` 和 `trap.c`



```

xv6 traps bttest Ubuntu
$ cd /mnt/d/xv6-labs/xv6-labs-2025-traps
$ make qemu
xv6 kernel is booting
init: starting sh
$ bttest
backtrace:
0x0000000080001e98
0x0000000080001d7a
0x0000000080001abc

```

2.3 保存进程的闹钟状态

进程结构增加间隔，累计 tick，处理程序地址，运行标志和一份完整 `trapframe`

```

int alarm_interval;
int alarm_ticks;
uint64 alarm_handler;
int alarm_active;
struct trapframe alarm_trapframe;

```

`sigalarm` 保存用户设置，间隔为 0 时自然停用，不能用处理程序地址是否为 0 判断是否启用，因为测试程序的处理函数可能正好位于地址 0

```

uint64
sys_sigalarm(void)
{

```

```

int ticks;
uint64 handler;
struct proc *p = myproc();

argint(0, &ticks);
argaddr(1, &handler);
if(ticks < 0)
    return -1;
p->alarm_interval = ticks;
p->alarm_ticks = 0;
p->alarm_handler = handler;
p->alarm_active = 0;
return 0;
}

```

2.4 定时触发和现场恢复

只有用户进程收到时钟中断时才累计 tick，间隔到达后先复制全部寄存器，再把返回用户态的 epc 改为处理程序地址，alarm_active 防止处理程序尚未结束时再次覆盖现场

```

if(which_dev == 2 && p->alarm_interval > 0 && !p->alarm_active){
    p->alarm_ticks++;
    if(p->alarm_ticks >= p->alarm_interval){
        p->alarm_ticks = 0;
        p->alarm_active = 1;
        memmove(&p->alarm_trapframe, p->trapframe,
                sizeof(struct trapframe));
        p->trapframe->epc = p->alarm_handler;
    }
}

```

处理程序最后调用 sigreturn，内核恢复完整 trapframe，并把原来的 a0 作为系统调用返回值，避免系统调用分发代码再次写坏恢复后的 a0

```

uint64
sys_sigreturn(void)
{
    struct proc *p = myproc();
    uint64 a0 = p->alarm_trapframe.a0;

    memmove(p->trapframe, &p->alarm_trapframe,
            sizeof(struct trapframe));
    p->alarm_active = 0; // 允许下一次触发
    return a0;
}

```

```
xv6 traps alarmtest Ubuntu
xv6 kernel is booting
hart 1 starting
hart 2 starting
init: starting sh
$ alarmtest
test0 start
.....alarm!
test0 passed
test1 start
.....alarm!
.....alarm!
.....alarm!
.....alarm!
.....alarm!
.....alarm!
.....alarm!
.....alarm!
.....alarm!
test1 passed
test2 start
.....alarm!
test2 passed
test3 start
test3 passed
```

2.5 整体测试

在实验仓库运行 `make grade`，汇编答案，回溯，四组 `alarmtest`，`usertests` 和 `time` 全部通过，得分 95/95

```
xv6 traps make grade Ubuntu
riscv64-linux-gnu-ld: warning: kernel/kernel has a LOAD segment with RWX permissions
riscv64-linux-gnu-objdump -S kernel/kernel > kernel/kernel.asm
riscv64-linux-gnu-objdump -t kernel/kernel | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$/d' >
make[1]: Leaving directory '/mnt/d/xv6-labs/xv6-labs-2025-traps'
== Test answers-traps.txt ==
answers-traps.txt: OK
== Test backtrace test ==
$ make qemu-gdb
backtrace test: OK (5.8s)
== Test running alarmtest ==
$ make qemu-gdb
(3.6s)
== Test alarmtest: test0 ==
alarmtest: test0: OK
== Test alarmtest: test1 ==
alarmtest: test1: OK
== Test alarmtest: test2 ==
alarmtest: test2: OK
== Test alarmtest: test3 ==
alarmtest: test3: OK
== Test usertests ==
$ make qemu-gdb
usertests: OK (49.7s)
== Test time ==
time: OK
Score: 95/95
```

三、实验中遇到的问题及解决方法

1. 最初打印 `%p` 时直接传入 `uint64`，编译器按格式检查认为参数类型不匹配，改为显式转换成 `void *`
2. 闹钟处理程序地址可能为 0，不能把地址 0 当成未注册，改为只依据间隔和运行标志判断

3. `sigreturn` 本身也是系统调用，如果固定返回 0，分发代码会把恢复后的 `a0` 覆盖，所以先保存原 `a0` 并把它作为返回值
4. 只恢复 `epc` 不能保证原程序继续正确运行，循环变量和临时值都在寄存器中，因此保存整个 `trapframe`

四、实验心得

这次实验把函数调用栈和 `trap` 联系了起来，栈回溯依靠编译器维护的帧指针，而用户级闹钟依靠内核保存 `trapframe` 并修改返回地址，两部分本质上都在读取和控制程序的执行现场

实现闹钟后也更清楚地看到，内核进入用户处理程序并不是普通函数调用，没有自动保存返回地址和通用寄存器，所有恢复工作都要由内核和 `sigreturn` 配合完成，少保存一个寄存器都可能让被中断程序出现很难定位的错误

实验五：Copy-on-Write

一、实验目的（要求）

本实验对应 MIT 6.810 Fall 2025 cow lab，官方页面为 <https://pdos.csail.mit.edu/6.828/2025/labs/cow.html>，目标是在 `fork()` 时共享物理页，只在父进程或子进程真正写入时复制，并完成以下内容

1. 修改 `uvmcopy()`，父子页表暂时指向相同物理页
2. 使用 RISC-V PTE 软件保留位记录 COW 状态
3. 在写缺页时分配新页，复制内容并恢复写权限
4. 为物理页维护引用计数，最后一个引用消失时才回收
5. 让 `copyout()` 也能处理 COW 页
6. 通过 `cowtest`，`usertests -q` 和官方评分

二、实验内容（步骤或过程）

2.1 标记 COW 页并共享物理内存

使用 PTE 的 RSW 位保存 COW 标志

```
#define PTE_COW (1L << 8)
```

`uvmcopy()` 不再调用 `kalloc()` 复制每一页，原来可写的页清除 `PTE_W` 并加上 `PTE_COW`，只读代码页保持原权限，子页表映射成功后增加物理页引用计数

```
pa = PTE2PA(*pte);
flags = PTE_FLAGS(*pte);
if(flags & PTE_W){
    flags = (flags & ~PTE_W) | PTE_COW;
    *pte = PA2PTE(pa) | flags;
}
if(mappages(new, i, PGSIZE, pa, flags) != 0)
    goto err;
krefinc(pa); // 父子现在一起使用这页
```

2.2 物理页引用计数

引用计数数组按 $(pa - \text{KERNBASE}) / \text{PGSIZE}$ 索引，和空闲链表共用 `kmem.lock`，分配时置 1，共享时加 1，释放时先减 1，只有减到 0 才放回空闲链表

```
void
kfree(void *pa)
{
    acquire(&kmem.lock);
    int i = reindex((uint64)pa);
    if(kmem.ref[i] < 1)
        panic("kfree ref");
    kmem.ref[i]--;
    if(kmem.ref[i] > 0){
        release(&kmem.lock);
        return;
    }

    memset(pa, 1, PGSIZE);
    struct run *r = (struct run *)pa;
    r->next = kmem.freelist;
    kmem.freelist = r;
    release(&kmem.lock);
}
```

初始化空闲内存时先把计数设为 1，再调用 `kfree()` 归零，否则初始化阶段会出现负引用

2.3 处理写缺页

`vmfault()` 遇到已经映射的 COW 页时，若引用计数为 1，说明已经没有其他进程共享，只需恢复写权限，若引用数大于 1，则分配新页并复制内容

```
pa = PTE2PA(*pte);
if(krefcount(pa) == 1){
    *pte = (*pte | PTE_W) & ~PTE_COW;
    return pa;
}

mem = (uint64)kalloc();
if(mem == 0)
    return 0;
memmove((void *)mem, (void *)pa, PGSIZE);
flags = (PTE_FLAGS(*pte) | PTE_W) & ~PTE_COW;
*pte = PA2PTE(mem) | flags;
kfree((void *)pa); // 原页少了一个引用
return mem;
```

原本只读的代码页没有 `PTE_COW`，写入时不会进入复制分支，进程会按非法写入处理

2.4 适配 copyout

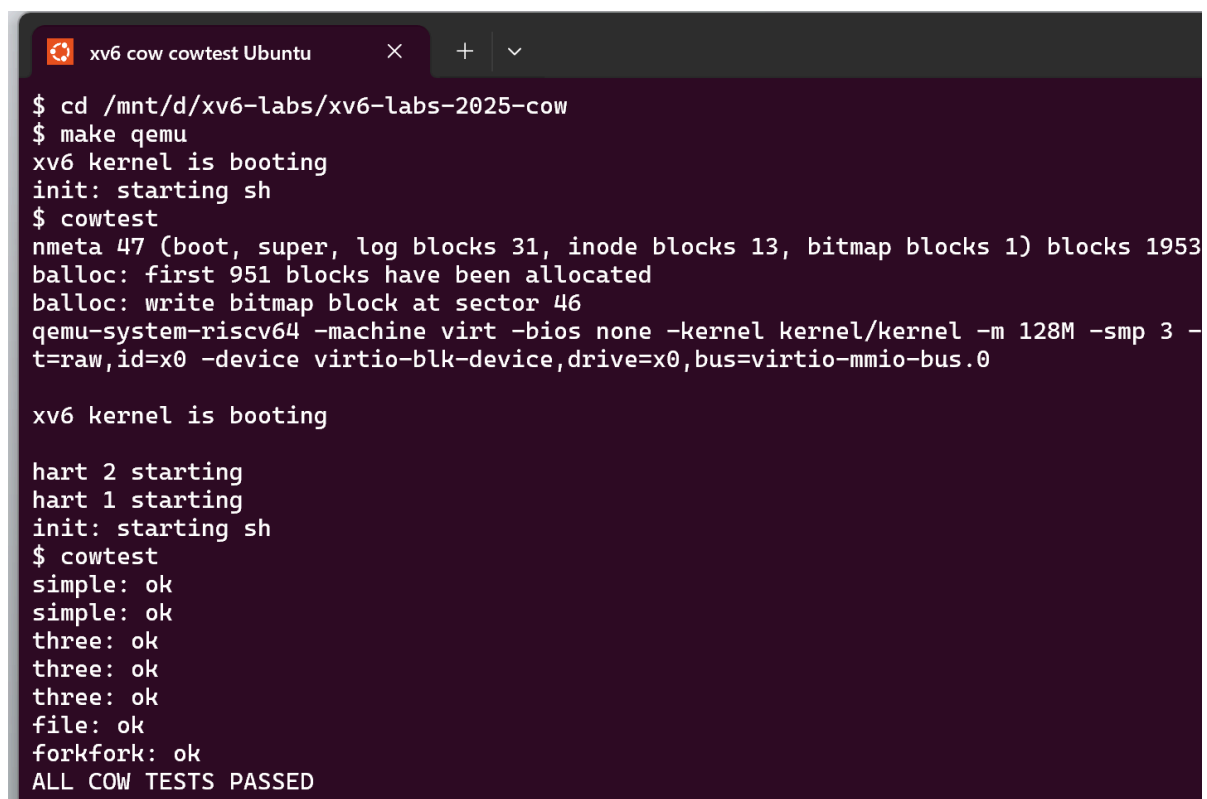
内核执行 `read()`, `pipe()` 等系统调用时会直接向用户页写数据, 不会产生用户态写缺页, 因此 `copyout()` 需要主动检查 COW 标志

```
pte = walk(pagetable, va0, 0);
if(pte && (*pte & PTE_COW)){
    if(vmfault(pagetable, va0, 0) == 0)
        return -1;
}
pa0 = walkaddr(pagetable, va0);

// 普通只读页仍然不能写
pte = walk(pagetable, va0, 0);
if((*pte & PTE_W) == 0)
    return -1;
```

2.5 运行结果

`cowtest` 的 `simple`, `three`, `file` 和 `forkfork` 全部通过, 说明大地址空间 `fork`, 多进程写入, 文件读入到 COW 页, 以及反复 `fork` 都能正常工作



```
xv6 cow cowtest Ubuntu
$ cd /mnt/d/xv6-labs/xv6-labs-2025-cow
$ make qemu
xv6 kernel is booting
init: starting sh
$ cowtest
nmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks 1953
ballocc: first 951 blocks have been allocated
ballocc: write bitmap block at sector 46
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -t=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ cowtest
simple: ok
simple: ok
three: ok
three: ok
three: ok
file: ok
forkfork: ok
ALL COW TESTS PASSED
```

官方 `make grade` 进一步检查 `copyin`, `copyout` 和全部 `usertests`, 得分 130/130

```
xv6 cow make grade Ubuntu
riscv64-linux-gnu-objdump -S kernel/kernel > kernel/kernel.asm
riscv64-linux-gnu-objdump -t kernel/kernel | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$/d' >
make[1]: Leaving directory '/mnt/d/xv6-labs/xv6-labs-2025-cow'
== Test running cowtest ==
$ make qemu-gdb
(28.5s)
== Test simple ==
simple: OK
== Test three ==
three: OK
== Test file ==
file: OK
== Test forkfork ==
forkfork: OK
== Test usertests ==
$ make qemu-gdb
(44.3s)
== Test usertests: copyin ==
usertests: copyin: OK
== Test usertests: copyout ==
usertests: copyout: OK
== Test usertests: all tests ==
usertests: all tests: OK
== Test time ==
time: OK
Score: 130/130
```

三、实验中遇到的问题及解决方法

1. 初始化引用数组时空闲页尚未处于已分配状态，直接 `kfree()` 会把计数减成负数，解决方法是在 `freerange()` 中先置 1
2. 只看 `PTE_W` 无法区分原本只读页和为了 COW 临时只读的页，因此增加 `PTE_COW` 标志
3. 每次写都复制会浪费内存，当引用计数已经为 1 时直接修改 PTE 权限即可
4. 用户写入会产生 trap，但内核 `copyout()` 不会经过同一路径，所以需要显式调用相同的 COW 处理逻辑

四、实验心得

Copy-on-Write 实验让我真正理解了操作系统如何通过“延迟复制”降低 `fork()` 的成本。传统做法在 `fork` 时立即为子进程复制全部用户页面，而 COW 让父子进程先共享同一批物理页，并把原本可写的页暂时设置为只读并标记 `PTE_COW`，只有某一方真正写入时才发生复制。这个机制让我体会到性能优化并不一定需要更复杂的算法，有时关键在于避免尚未发生的工作。实验中最重要另一个概念是物理页引用计数：每增加一个共享映射就必须增加计数，每取消一个映射就必须减少计数，只有最后一个引用消失后才能真正 `kfree`。引用计数错误可能导致两类严重问题，一类是页面被过早释放形成悬空引用，另一类是页面永远无法回收造成内存泄漏。处理写缺页时，还需要区分原本只读的代码页和为了 COW 临时变成只读的页面，因此单纯检查 `PTE_W` 并不够。`copyout()` 也让我意识到内核向用户空间写数据时并不会走普通的用户写缺页路径，所以必须显式复用 COW 处理逻辑。通过本实验，我对页表权限、异常处理和物理页生命周期之间的配合有了更深入的认识。

实验六：Networking

一、实验目的（要求）

本实验对应 MIT 6.1810 Fall 2025 networking lab，官方页面为 <https://pdos.csail.mit.edu/6.828/2025/labs/net.html>，目标是完成 QEMU E1000 网卡驱动和 UDP 接收路径

1. 补全发送和接收描述符环

2. 正确回收发送缓冲区并补充接收缓冲区
3. 解析 Ethernet, IPv4 和 UDP 头部, 处理网络字节序
4. 实现 bind 和阻塞式 `recv`, 每个端口最多排队 16 个包
5. 通过 txone, ARP, IP, ping, DNS, free 和官方评分

二、实验内容（步骤或过程）

2.1 E1000 发送环

发送时读取 `E1000_TDT` 找到网卡期待的描述符, `DD` 未置位说明该槽仍在被使用, 驱动返回失败, 槽位可用时先释放上一次发送完成的缓冲区, 再填写地址, 长度和命令位

```
acquire(&e1000_lock);
uint32 i = regs[E1000_TDT] % TX_RING_SIZE;
if((tx_ring[i].status & E1000_TXD_STAT_DD) == 0){
    release(&e1000_lock);
    return -1;
}
if(tx_ring[i].addr)
    kfree((void *)tx_ring[i].addr);

tx_ring[i].addr = (uint64)buf;
tx_ring[i].length = len;
tx_ring[i].cmd = E1000_TXD_CMD_EOP | E1000_TXD_CMD_RS;
tx_ring[i].status = 0;
regs[E1000_TDT] = (i + 1) % TX_RING_SIZE;
release(&e1000_lock);
```

2.2 E1000 接收环

接收中断可能对应多个包, 因此循环处理 `RDT + 1`, 先分配新缓冲区放回描述符, 再释放接收锁并把旧缓冲区交给协议栈, 这样 ARP 处理过程中调用发送函数不会和接收锁互相等待

```
uint32 i = (regs[E1000_RDT] + 1) % RX_RING_SIZE;
if((rx_ring[i].status & E1000_RXD_STAT_DD) == 0)
    break;

char *oldbuf = (char *)rx_ring[i].addr;
int len = rx_ring[i].length;
char *newbuf = kalloc();
if(newbuf == 0)
    panic("e1000 recv");
rx_ring[i].addr = (uint64)newbuf;
rx_ring[i].status = 0;
regs[E1000_RDT] = i;

net_rx(oldbuf, len); // 描述符已经重新可用
```

2.3 UDP 端口队列

使用固定数组保存已绑定端口，每个端口有独立自旋锁，链表队头，队尾和包数量，队列上限为 16，避免某个端口占完内存

```
struct udp_queue {
    struct spinlock lock;
    int used;
    uint16 port;
    int count;
    struct udp_packet *head;
    struct udp_packet *tail;
};
```

`sys_recv()` 在队列为空时使用队列地址作为等待通道，有包到达后取出最早的节点，再把源 IP，源端口和负载复制到用户空间

```
acquire(&q->lock);
while(q->head == 0){
    if(killed(myproc())){
        release(&q->lock);
        return -1;
    }
    sleep(q, &q->lock);
}
struct udp_packet *pkt = q->head;
q->head = pkt->next;
if(q->head == 0)
    q->tail = 0;
q->count--;
release(&q->lock);
```

2.4 IP 和 UDP 解析

先检查 IPv4 版本，IP 头长度和协议号，再用 `ntohs` 与 `ntohl` 转换端口，长度和地址，目标端口没有绑定或队列已满时释放包

```
int ihl = (ip->ip_vhl & 0xf) * 4;
if((ip->ip_vhl >> 4) != 4 || ihl < sizeof(struct ip) ||
    ip->ip_p != IPPROTO_UDP){
    kfree(buf);
    return;
}

struct udp *udp = (struct udp *)((char *)ip + ihl);
int dport = ntohs(udp->dport);
int ulen = ntohs(udp->ulen);
```

入队完成后调用 `wakeup(q)`，等待这个端口的 `recv` 会重新检查队列并继续运行

2.5 运行结果

实际终端中同时运行宿主端 `nettest.py grade` 和 xv6 的 `nettest grade`, txone, ARP, IP, 四组 ping, DNS 和缓冲区释放都成功

```
xv6 net nettest Ubuntu
$ cd /mnt/d/xv6-labs/xv6-labs-2025-net
$ python3 nettest.py grade # 同一 Ubuntu 环境中的宿主端
$ make qemu
xv6 kernel is booting
init: starting sh
$ nettest grade
txone: sending one packet
arp_rx: received an ARP packet
ip_rx: received an IP packet
ping0: starting
ping0: OK
ping1: starting
ping1: OK
ping2: starting
ping2: OK
ping3: starting
ping3: OK
dns: starting
DNS arecord for pdos.csail.mit.edu. is 128.52.129.126
dns: OK
free: OK

宿主端结果
txone: OK
```

官方 `make grade` 全部通过, 得分 171/171

```
xv6 net make grade Ubuntu
riscv64-linux-gnu-objdump -t kernel/kernel | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$/d' >
make[1]: Leaving directory '/mnt/d/xv6-labs/xv6-labs-2025-net'
== Test running nettest ==
$ make qemu-gdb
(17.5s)
== Test nettest: txone ==
nettest: txone: OK
== Test nettest: arp_rx ==
nettest: arp_rx: OK
== Test nettest: ip_rx ==
nettest: ip_rx: OK
== Test nettest: ping0 ==
nettest: ping0: OK
== Test nettest: ping1 ==
nettest: ping1: OK
== Test nettest: ping2 ==
nettest: ping2: OK
== Test nettest: ping3 ==
nettest: ping3: OK
== Test nettest: dns ==
nettest: dns: OK
== Test nettest: free ==
nettest: free: OK
== Test time ==
time: OK
Score: 171/171
```

三、实验中遇到的问题及解决方法

1. 发送缓冲区不能在提交描述符后立即释放，必须等网卡设置 `DD`，否则 DMA 可能读取已经被复用的页面
2. 接收描述符交出旧缓冲区后必须马上补入新页面，否则环绕后网卡会写入协议栈仍在使用的内存
3. 如果接收过程一直持有和发送相同的锁，ARP 接收调用发送回复时会自锁，解决方法是分开收发锁，并在调用 `net_rx()` 前释放接收锁
4. 网络头采用大端顺序，RISC-V 是小端顺序，多字节字段都要转换

四、实验心得

本次网络实验让我看到操作系统中设备驱动、DMA 缓冲区和协议栈是如何连接起来的。E1000 驱动部分最关键的不是简单填写发送和接收描述符，而是明确缓冲区在每个时刻的所有权。发送时，缓冲区交给网卡后不能立即释放，必须等硬件设置完成状态后才能回收，否则 DMA 仍可能访问已经被重新使用的内存；接收时，把旧缓冲区交给上层协议栈之前又必须先给描述符补上新的页面，避免网卡下一次接收数据时覆盖协议栈正在处理的内容。这些细节让我体会到驱动程序实际上是在协调 CPU 与硬件异步工作的边界。继续实现 Ethernet、IPv4 和 UDP 解析后，我对网络字节序、头部长度和端口字段有了更实际的认识。bind 和阻塞式 recv 则把网络接收与进程睡眠/唤醒机制结合起来：队列为空时进程休眠，有数据包入队后再 wakeup，对每个端口还要限制队列长度并正确释放无效包。通过 txone、ARP、IP、ping、DNS 等测试，我看到一个看似很小的 UDP 功能实际上跨越了硬件描述符、协议解析、同步机制和用户接口多个层次。本实验使我更加理解操作系统 I/O 的核心问题是所有权、并发和资源回收。

实验七：Locks

一、实验目的（要求）

本实验对应 MIT 6.1810 Fall 2025 locks lab，官方页面为 <https://pdos.csail.mit.edu/6.828/2025/labs/lock.html>，2025 版本的第二部分是读写自旋锁，与参考报告旧版本中的块缓存任务不同，本次以官方当前要求为准

1. 把单一物理页空闲链表改为每 CPU 一个链表
2. 本地链表为空时从其他 CPU 取页
3. 降低 `kmem` 锁的 test-and-set 争用次数
4. 实现允许多个读者并发，写者独占，并且写者优先的读写自旋锁
5. 通过 `kallocstest`，`rwktest`，`sbrkmuch`，`usertests` 和官方评分

二、实验内容（步骤或过程）

2.1 每 CPU 空闲链表

为每个 CPU 保存独立锁和链表，所有锁名以 `kmem` 开头，启动时把空闲页轮流分给各 CPU，减少第一轮分配时集中访问 CPU 0

```

struct {
    struct spinlock lock;
    struct run *freelist;
} kmem[NCPU];

void
kinit(void)
{
    for(int i = 0; i < NCPU; i++)
        initlock(&kmem[i].lock, "kmem");
    freerange(end, (void *)PHYSTOP);
}

```

`kfree()` 关闭中断后读取 `cpuid()`，把页面放到当前 CPU 的链表，`push_off()` 保证操作期间不会迁移到另一个 CPU

```

push_off();
int id = cpuid();
acquire(&kmem[id].lock);
r->next = kmem[id].freelist;
kmem[id].freelist = r;
release(&kmem[id].lock);
pop_off();

```

2.2 本地取页和跨 CPU 取页

`ka1loc()` 先访问本地链表，本地为空时再检查其他 CPU，跨 CPU 操作只修改链表头，不在远端锁内遍历链表，这样临界区很短，也不会把供给方的页面一次拿空

```

for(int n = 1; n < NCPU && r == 0; n++){
    int donor = (id + n) % NCPU;
    acquire(&kmem[donor].lock);
    r = kmem[donor].freelist;
    if(r)
        kmem[donor].freelist = r->next; // 锁里只改链表头
    release(&kmem[donor].lock);
}

```

每次跨 CPU 只取一页，虽然会增加无争用的加锁次数，但 test-and-set 统计只在真正争用时增加，短临界区让 `test4` 保持在限制以内，同时 `countfree` 和 `sbrkmuch` 仍可使用全部物理内存

```
xv6 lock kalloctest Ubuntu
start test3
.....child done 10000

test3 OK
start test4
.....child done 100000
.child done 100000
.child done 100000
--- lock kmem stats
lock: kmem: #test-and-set 3391 #acquire() 628483
lock: kmem: #test-and-set 13209 #acquire() 783630
lock: kmem: #test-and-set 9937 #acquire() 780531
lock: kmem: #test-and-set 4195 #acquire() 1540899
lock: kmem: #test-and-set 437 #acquire() 108972
lock: kmem: #test-and-set 0 #acquire() 104881
lock: kmem: #test-and-set 2 #acquire() 100790
lock: kmem: #test-and-set 2 #acquire() 96699
--- top 5 contended locks:
lock: wait_lock: #test-and-set 74199363 #acquire() 40015
lock: proc: #test-and-set 2026257 #acquire() 890149
lock: proc: #test-and-set 193540 #acquire() 1760287
lock: proc: #test-and-set 126000 #acquire() 2283142
lock: uart: #test-and-set 84939 #acquire() 1547
tot= 31173
```

2.3 写者优先的读写自旋锁

读写锁保存读者数，活动写者和等待写者数，`guard` 只保护这些字段的短暂更新，读锁返回以后不会继续占用 `guard`，因此多个读者仍可并发执行

```
struct rwspinlock {
    uint guard;
    uint readers;
    uint writer;
    uint waiting;
};
```

读者进入保护区后同时检查活动写者和等待写者，条件满足才增加读者数，等待写者数不为零时，新读者会在外面重试

```
for(;;){
    rwguard_acquire(rwlk);
    if(rwlk->writer == 0 && rwlk->waiting == 0){
        rwlk->readers++;
        rwguard_release(rwlk);
        return;
    }
    rwguard_release(rwlk);
}
```

写者先增加 `waiting`，从这一刻开始新读者不能进入，再等待已有读者退出，获取成功时减少等待数并设置活动写者状态

```
rwguard_acquire(rwlk);
rwlk->waiting++;
rwguard_release(rwlk);
```

```

for(;;){
    rwguard_acquire(rwlk);
    if(rwlk->writer == 0 && rwlk->readers == 0){
        rwlk->waiting--;
        rwlk->writer = 1;
        rwguard_release(rwlk);
        return;
    }
    rwguard_release(rwlk);
}

```

```

xv6 lock rwlctest Ubuntu
rwspinlock_test: step 10: prepare read_acquire for multiple writer priority test
rwspinlock_test: step 11: multiple writer priority test
rwspinlock_test: step 12: prepare read_acquire for multiple writer priority test
rwspinlock_test: step 13: multiple writer priority test
rwspinlock_test: step 14: prepare read_acquire for multiple writer priority test
rwspinlock_test: step 15: multiple writer priority test
rwspinlock_test: step 16: prepare read_acquire for multiple writer priority test
rwspinlock_test: step 17: multiple writer priority test
rwspinlock_test: step 18: prepare read_acquire for multiple writer priority test
rwspinlock_test: step 19: multiple writer priority test
rwspinlock_test: step 20: prepare read_acquire for multiple writer priority test
rwspinlock_test: step 21: multiple writer priority test
rwspinlock_test: step 22: prepare read_acquire for multiple writer priority test
rwspinlock_test: step 23: multiple writer priority test
rwspinlock_test: step 24: prepare read_acquire for multiple writer priority test
rwspinlock_test: step 25: multiple writer priority test
rwspinlock_test: step 26: prepare read_acquire for multiple writer priority test
rwspinlock_test: step 27: multiple writer priority test
rwspinlock_test: step 28: prepare read_acquire for multiple writer priority test
rwspinlock_test: step 29: multiple writer priority test
rwspinlock_test: step 30: done
rwspinlock_test(0): 0
rwspinlock_test(3): 0
rwspinlock_test(2): 0
rwspinlock_test(1): 0

```

2.4 整体测试

官方 `make grade` 中 `kalloc_test` 四项, `sbrkmuch`, `rwlctest`, `usertests` 和 `time` 全部通过, 得分 100/100

```
xv6 lock make grade Ubuntu
make[1]: Leaving directory '/mnt/d/xv6-labs/xv6-labs-2025-lock'
== Test running kalloc test ==
$ make qemu-gdb
(67.7s)
== Test kalloc test: test1 ==
kalloc test: test1: OK
== Test kalloc test: test2 ==
kalloc test: test2: OK
== Test kalloc test: test3 ==
kalloc test: test3: OK
== Test kalloc test: test4 ==
kalloc test: test4: OK
== Test kalloc test: sbrkmuch ==
$ make qemu-gdb
kalloc test: sbrkmuch: OK (7.1s)
== Test running rwlk test ==
$ make qemu-gdb
(11.6s)
== Test rwlk test ==
rwlk test: OK
== Test user tests ==
$ make qemu-gdb
user tests: OK (128.7s)
== Test time ==
time: OK
Score: 100/100
```

三、实验中遇到的问题及解决方法

1. 在远端锁内遍历并拆分链表时，临界区太长，test4 的 test-and-set 次数偶尔超限，改成只取链表头后争用稳定下降
2. 一次拿走供给方全部页面会让另一个 CPU 立刻反向取页，还可能使大块申请暂时失败，单页转移没有这个问题
3. 最初用多个原子字段分别检查写者状态，压力测试中仍有很短的竞争窗口，后来用内部 guard 统一更新状态，连续复测后结果稳定
4. guard 不能覆盖整个读临界区，否则读锁会退化成普通互斥锁，所以读者数增加后马上释放它

四、实验心得

Locks实验让我对锁除正确性外的性能与公平性问题有了更深认识。物理页分配器最初用全局空闲链表和一把锁，多核操作时会频繁竞争，拆分出每个CPU对应的空闲链表后，多数操作只需访问本地状态，仅本地为时空时才从其他CPU取页，明显缩短了共享临界区，这让我明白，降低锁竞争不能只盲目增加锁的数量，更要按访问局部性对数据分区。实验中还发现，跨CPU取页一次拿太多会造成新的不平衡，在远端锁内过多遍历会增加争用，因此临界区要尽可能短。写者优先的读写自旋锁让我认识到，并发控制除了保证互斥正确，还要考虑公平性，允许多读者并行能提高并发度，但如果新读者不断进入，写者可能长期得不到执行，所以需要记录等待的写者，让后续读者让行；内部guard只需短暂保护相关计数与状态，不能覆盖整个读临界区，否则读写锁会退化为普通互斥锁。经过本次实验，我对锁粒度、数据分区、临界区长度和饥饿问题的关系有了更具体的理解。

实验八：File system

一、实验目的（要求）

本实验对应 MIT 6.1810 Fall 2025 file system lab，官方页面为 <https://pdos.csail.mit.edu/6.828/2025/labs/fs.html>，目标是扩展 xv6 文件寻址能力并实现符号链接

1. 在不改变磁盘 inode 大小的条件下增加二级间接块

2. 把最大文件扩大到 65803 个数据块
3. 在截断文件时释放所有直接块，一级间接块和二级间接块
4. 增加 `symlink` 系统调用，`T_SYMLINK` 和 `O_NOFOLLOW`
5. 递归跟随符号链接，并检测循环
6. 通过 `bigfile`, `symlinktest`, `usertests` 和官方评分

二、实验内容（步骤或过程）

2.1 inode 地址布局

把直接块数量从 12 改为 11，原来的第 12 项继续保存一级间接块，第 13 项保存二级间接块，总地址项数量不变

```
#define NDIRECT 11
#define NINDIRECT (BSIZE / sizeof(uint))
#define MAXFILE (NDIRECT + NINDIRECT + NINDIRECT * NINDIRECT)

struct dinode {
    short type;
    short major;
    short minor;
    short nlink;
    uint size;
    uint addrs[NDIRECT + 2];
};
```

内存 inode 的 `addrs` 也改为相同长度，保证磁盘和内存结构一致

2.2 二级间接块映射

逻辑块号先减去 11 个直接块和 256 个一级间接块，商选择二级块中的一级间接表，余数选择最终数据块

```
bn -= NINDIRECT;
uint first = bn / NINDIRECT;
uint second = bn % NINDIRECT;

bp = bread(ip->dev, ip->addrs[NDIRECT + 1]);
a = (uint *)bp->data;
if((addr = a[first]) == 0){
    addr = balloc(ip->dev);
    if(addr){
        a[first] = addr;
        log_write(bp);
    }
}
brelse(bp);
```

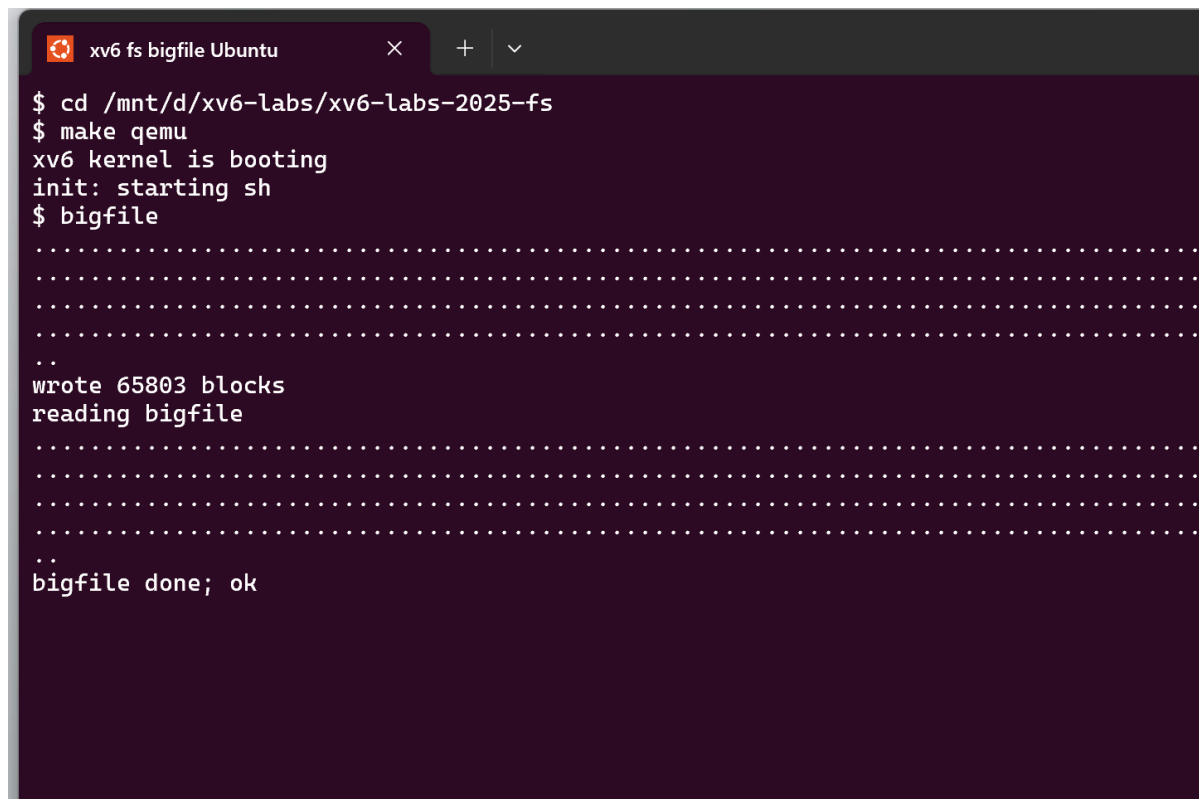
随后读取选中的一级间接表，按 `second` 分配或返回数据块，每个修改过的地址块都写入日志，并及时 `brelse()`

2.3 截断大文件

`itrunc()` 遍历二级块中的每个一级间接表，先释放其中的数据块，再释放一级表，最后释放最外层二级块

```
for(i = 0; i < NINDIRECT; i++){
    if(a[i]){
        struct buf *bp2 = bread(ip->dev, a[i]);
        uint *a2 = (uint *)bp2->data;
        for(j = 0; j < NINDIRECT; j++){
            if(a2[j])
                bfree(ip->dev, a2[j]);
        }
        brelse(bp2);
        bfree(ip->dev, a[i]);
    }
}
```

`bigfile` 实际写入 65803 个块，再逐块读回验证内容



```
xv6 fs bigfile Ubuntu
$ cd /mnt/d/xv6-labs/xv6-labs-2025-fs
$ make qemu
xv6 kernel is booting
init: starting sh
$ bigfile
.....
..
wrote 65803 blocks
reading bigfile
.....
..
bigfile done; ok
```

2.4 创建符号链接

`sys_symlink()` 使用 `create()` 建立 `T_SYMLINK` inode，把目标路径连同结尾的零字节写入 inode 数据区，目标暂时不存在也能创建成功

```

if((ip = create(path, T_SYMLINK, 0, 0)) == 0){
    end_op();
    return -1;
}
int n = strlen(target) + 1;
if(writei(ip, 0, (uint64)target, 0, n) != n){
    iunlockput(ip);
    end_op();
    return -1;
}

```

2.5 open 跟随链接

普通 `open()` 遇到符号链接时读取目标路径，释放当前 inode，再调用 `namei()` 查找目标，最多跟随 10 层，达到上限视为循环，带 `O_NOFOLLOW` 时直接打开链接 inode

```

if((omode & O_NOFOLLOW) == 0){
    for(int depth = 0; ip->type == T_SYMLINK; depth++){
        if(depth == 10 || readi(ip, 0, (uint64)path,
                                0, MAXPATH) < 1){
            iunlockput(ip);
            end_op();
            return -1;
        }
        path[MAXPATH - 1] = 0;
        iunlockput(ip);
        if((ip = namei(path)) == 0){
            end_op();
            return -1;
        }
        ilock(ip);
    }
}

```


4. 符号链接循环不能无限递归，使用最多 10 层的限制返回错误

四、实验心得

本次文件系统实验让我更清晰理解了inode中块地址到实际磁盘数据的映射。要在不扩大inode结构的前提下支持更大文件，需要减少一个直接块地址，将腾出的地址项作为二级间接块入口，逻辑块号经过两级索引得到数据块，最大文件容量也从线性扩展变为更大的平方级寻址范围。在实现bmap和itrunc时，我认识到分配和释放必须保持严格层次关系：地址块修改后要写入日志，bread()与brelse()要正确配对，截断文件必须从内层数据块逐级向外释放，否则丢失索引结构就找不到内部数据。符号链接部分让我理解了“名称”和“对象”的区别，符号链接inode保存目标路径而非目标inode，因此目标可以后创建也能删除后重现，open()每次都需要重新解析路径，还要支持O_NOFOLLOW，为了避免循环链接导致无限递归，需要限制最大跟随深度。完成大文件和符号链接两部分内容后，我认识到文件系统设计既要考虑容量扩展，也要保证元数据、日志和路径解析的一致性。

实验九：mmap

一、实验目的（要求）

本实验对应 MIT 6.810 Fall 2025 mmap lab，官方页面为 <https://pdos.csail.mit.edu/6.828/2025/lab/s/mmap.html>，目标是实现文件到用户地址空间的惰性映射

1. 增加 mmap 和 munmap 系统调用
2. 使用固定大小 VMA 数组记录地址，长度，权限，映射方式和文件
3. 首次访问时才分配物理页并从文件读取
4. MAP_SHARED 解除映射和进程退出时写回文件
5. 支持从 VMA 开头或末尾进行部分解除映射
6. fork 时复制 VMA 并保持文件引用
7. 通过 mmaptest, usertests 和官方评分

二、实验内容（步骤或过程）

2.1 VMA 数据结构

每个进程保存 16 个 VMA，记录虚拟地址范围，文件偏移，权限，标志和文件指针

```
#define NVMA 16
struct vma {
    uint64 addr;
    uint64 len;
    uint64 offset;
    int prot;
    int flags;
    struct file *file;
    int used;
};
```

映射区从独立的 MMAPBASE 开始，不扩大进程堆的 p->sz，这样 munmap 后再次访问不会被普通 lazy allocation 当作堆页重新分配

```
#define MMAPBASE (1L << 32)

uint64 addr = MMAPBASE;
for(int j = 0; j < NVMA; j++)
    if(p->vmass[j].used &&
        addr < p->vmass[j].addr + p->vmass[j].len)
        addr = p->vmass[j].addr + p->vmass[j].len;
addr = PGROUNDUP(addr);
```

`mmap()` 只登记 VMA 并调用 `filedup()` 增加引用，不分配页面，也不读取文件

2.2 惰性装页

读写缺页先查找覆盖故障地址的 VMA，检查访问权限，分配并清零页面，再按 VMA 内偏移从 inode 读取数据

```
uint64 page = PGROUNDDOWN(va);
char *mem = kalloc();
if(mem == 0)
    return 0;
memset(mem, 0, PGSIZE);

ilock(v->file->ip);
int n = readi(v->file->ip, 0, (uint64)mem,
              v->offset + page - v->addr, PGSIZE);
iunlock(v->file->ip);
```

PTE 权限由 `PROT_READ`，`PROT_WRITE` 和 `PROT_EXEC` 转换，RISC-V 可写页同时设置可读位，映射成功后重新执行产生缺页的用户指令

```
int perm = PTE_U;
if(v->prot & (PROT_READ | PROT_WRITE))
    perm |= PTE_R;
if(v->prot & PROT_WRITE)
    perm |= PTE_W;
if(v->prot & PROT_EXEC)
    perm |= PTE_X;
mappages(p->pagetable, page, PGSIZE, (uint64)mem, perm);
```

2.3 解除映射和共享写回

`vmaunmap()` 允许删除整个 VMA，开头部分或末尾部分，只处理实际存在的 PTE，未访问过的惰性页面无需分配

`MAP_SHARED` 页面在取消映射前写回 inode，写回长度不超过文件原有大小，避免映射到文件末尾之外的零填充部分意外扩展文件

```
if(v->flags == MAP_SHARED){
    uint64 off = v->offset + a - v->addr;
    uint n = 0;
    ilock(v->file->ip);
    if(off < v->file->ip->size){
        n = v->file->ip->size - off;
```

```

    if(n > PGSIZE)
        n = PGSIZE;
}
iunlock(v->file->ip);

if(n){
    begin_op();
    ilock(v->file->ip);
    writei(v->file->ip, 0, PTE2PA(*pte), off, n);
    iunlock(v->file->ip);
    end_op();
}
}

```

整个 VMA 删除后调用 `fclose()`，部分删除则调整 `addr`，`offset` 和 `len`

2.4 exit 和 fork

进程退出时逐个调用 `vmaunmap()`，共享修改会先写回，所有页面和文件引用都被释放，fork 后子进程复制 VMA 描述并为每个文件增加引用

```

for(i = 0; i < NVMA; i++){
    if(p->vmass[i].used){
        np->vmass[i] = p->vmass[i];
        np->vmass[i].file = filedup(p->vmass[i].file);
    }
}

```

映射区不属于 `p->sz`，所以 fork 不提前复制 mmap 页面，父子进程以后分别缺页并从同一文件读取，符合实验允许不共享物理页的要求

2.5 运行结果

`mmaptest` 的基本映射，私有映射，只读检查，共享读写，脏页写回，惰性访问，双文件，fork，解除映射后禁止访问和只读页禁止写入全部成功

```
xv6 mmap mmaptest Ubuntu × + ▾
test mmap private: OK
test mmap read-only
test mmap read-only: OK
test mmap read/write
test mmap read/write: OK
test mmap dirty
test mmap dirty: OK
test not-mapped unmap
test not-mapped unmap: OK
test lazy access
test lazy access: OK
test mmap two files
test mmap two files: OK
test fork
test fork: OK
test munmap prevents access
usertrap(): unexpected scause 0xd pid=5
             sepc=0xa2a stval=0x100005000
usertrap(): unexpected scause 0xd pid=6
             sepc=0xab2 stval=0x100004000
test munmap prevents access: OK
test writes to read-only mapped memory
usertrap(): unexpected scause 0xf pid=7
             sepc=0xbfa stval=0x100004000
test writes to read-only mapped memory: OK
mmaptest: all tests succeeded
```

官方 `make grade` 进一步运行 `usertests`, 全部通过, 得分 170/170

```
xv6 mmap make grade Ubuntu × + ▾
== Test  mmaptest: mmap private ==
mmaptest: mmap private: OK
== Test  mmaptest: mmap read-only ==
mmaptest: mmap read-only: OK
== Test  mmaptest: mmap read/write ==
mmaptest: mmap read/write: OK
== Test  mmaptest: mmap dirty ==
mmaptest: mmap dirty: OK
== Test  mmaptest: not-mapped unmap ==
mmaptest: not-mapped unmap: OK
== Test  mmaptest: lazy access ==
mmaptest: lazy access: OK
== Test  mmaptest: two files ==
mmaptest: two files: OK
== Test  mmaptest: fork_test ==
mmaptest: fork_test: OK
== Test  mmaptest: munmap_noaccess ==
mmaptest: munmap_noaccess: OK
== Test  mmaptest: read_only_write ==
mmaptest: read_only_write: OK
== Test usertests ==
$ make qemu-gdb
usertests: OK (47.7s)
== Test time ==
time: OK
Score: 170/170
```

三、实验中遇到的问题及解决方法

1. 第一版把 `mmap` 区放在 `p->sz` 内, 解除映射后普通 `vmfault()` 又把地址当成惰性堆页补回, 导致非法访问没有终止, 改用独立高地址映射区
2. `MAP_SHARED` 写回整个最后一页会把文件扩展到页边界, 按 `inode` 当前大小截断写回长度
3. 未访问页面没有 PTE, `munmap` 必须允许跳过这些页面, 不能把惰性映射当作错误
4. 文件描述符关闭后映射仍要有效, 因此 VMA 建立时使用 `filedup()`, 删除时再 `fileclose()`

四、实验心得

本次mmap实验将此前学习的文件系统、页表与缺页处理整合在了一起。mmap()不会立即读取完整文件，只会建立VMA，记录相关地址范围、长度等相关信息；当程序首次访问页面时，内核才会根据缺页地址找到对应VMA，分配物理页并读取数据。这种惰性装页帮助我区分了“地址空间存在对应区域”与“该区域已分配物理页”是两个不同概念。实现munmap时，未访问页面可能不存在PTE，不能将无页面判定为错误；针对MAP_SHARED，还需要在取消映射或进程退出前将修改写回文件，同时要避免文件末尾因整页写回被意外扩展。VMA负责管理文件生命周期，文件描述符关闭后映射仍有效，因此建立映射时需调用filedup()，VMA完全删除后再调用fileclose()。fork时会复制VMA描述与文件引用，页面仍可继续按需缺页加载。通过本次实验，我更清晰地认识到，mmap不只是一个系统调用，而是整合了虚拟内存、文件I/O等内容的一套完整机制。

总结

本系列实验从环境搭建和用户态工具出发，逐步深入系统调用、页表、陷阱、写时复制、网络、锁、文件系统和内存映射，完整覆盖了 xv6 中进程、内存、并发、I/O 与持久化等核心机制。十份报告既记录了功能实现，也保留了测试截图、问题定位和修复过程，形成了从“理解接口”到“修改内核”再到“验证系统行为”的连续实验链路。

贯穿各实验的共同认识是：操作系统依靠清晰的抽象和严格的资源生命周期维持正确性。页表、COW 与 mmap 体现了地址转换和按需分配；引用计数、锁与中断处理体现了并发环境中的共享状态管理；文件系统和网络实验则进一步说明，设备、缓存、磁盘块与数据包都需要明确的所有权和边界检查。通过持续运行单项测试、`usertests -q` 和 `make grade`，实验过程也强化了以可复现测试验证内核修改的习惯。